

Stefan Heiss

Technische Hochschule Ostwestfalen-Lippe

Fachbereich Elektrotechnik und Technische Informatik

Datensicherheit

01 – Grundlagen

Version vom 16. Mai 2025

Vorwort

Liebe Studierende,

herzlich Willkommen zum Modul Datensicherheit.

Das vorliegende erste Studienheft 01 zu diesem Modul führt mit einem *Historischer Abriss der Kryptographie* in das Thema ein. Gönnen sie sich einen „spielerischen“ Einstieg, indem sie parallel zur Lektüre von Kapitel 1 das unter <https://www.cryptool.org/de/> frei verfügbare *CrypTool 1* erproben und zur Lösung einiger Aufgaben dieses Kapitels nutzen.

Da sehr viele kryptographische Algorithmen mit Hilfe der Zahlbereiche

$$\mathbb{Z}_n = \{0, 1, 2, \dots, n-1\}$$

definiert sind (bei denen *modulo* n gerechnet wird), werden grundlegende Eigenschaften dieser Zahlbereiche in Kapitel 2 behandelt.

In Kapitel 3 werden Gefährdungen und damit Schutzziele klassifiziert, die bei einem Betrieb eines IT-Systems beachtet werden sollten. Auch werden in diesem Kapitel bereits auf einer recht allgemeinen Abstraktionsebene Maßnahmen genannt, die der Sicherstellung der zuvor formulierten Sicherheitsziele dienen können.

Das letzte Kapitel dieses Studienhefts widmet sich schließlich verschiedenen Aspekten von Verschlüsselungsverfahren.

In den weiteren Studienheften werden für alle in Kapitel 3 skizzierten IT-Sicherheitsmechanismen konkrete Verfahren und Algorithmen behandelt, die in aktuellen Protokollen und Anwendungen zum Einsatz kommen. Den Schwerpunkt von Studienheft 02 bilden dabei die symmetrischen kryptographischen Verfahren und den Schwerpunkt von Studienheft 03 die asymmetrischen Verfahren. In der Regel stehen Implementierungen kryptographischer Verfahren und Algorithmen in größeren Programmier- bzw. Laufzeitumgebungen zur Verfügung. Beispielsweise werden sie lernen, wie solche Implementierungen im Rahmen von Java-Programmen genutzt werden können.

Inhaltsverzeichnis

1 Historischer Abriss der Kryptographie	1
1.1 Kryptographie und Kryptoanalyse	2
1.2 Klassische Verschlüsselungsverfahren	3
1.3 Das One-Time-Pad: Ein absolut sicheres Verfahren	9
1.4 Das Hill-Verschlüsselungsverfahren: Ein erstes algebraisches Verfahren .	12
1.5 Elektromechanische Verschlüsselungsmaschinen	12
1.6 Moderne Verschlüsselungsverfahren	13
2 Modulare Arithmetik	15
2.1 Der Ring $\mathbb{Z}_n$	16
2.2 Inverse Elemente in $\mathbb{Z}_n$ bezüglich der Multiplikation	21
2.3 Matrizen über $\mathbb{Z}_n$	24
2.4 Der erweiterte Euklid'sche Algorithmus	26
2.5 Berechnung inverser Elemente in $\mathbb{Z}_n$	34
2.6 Mathematischer Exkurs: <i>Euler'sche φ-Funktion</i>	36
2.7 Primfaktorzerlegung natürlicher Zahlen	38
3 IT-Sicherheit: Gefährdungen und Maßnahmen	41
3.1 Vertraulichkeit (Confidentiality, Secrecy, Privacy)	42
3.1.1 Schutzmaßnahmen: Verschlüsselungsverfahren	43
3.2 Integrität (Integrity)	43
3.2.1 Schutzmaßnahmen: Hashfunktionen, Whitelists	44
3.3 Authentizität von Nachrichten (Message Authenticity)	45

3.3.1	Schutzmaßnahmen: Signaturen, Message Authentication Codes . . .	46
3.4	Authentizität von Nutzern (User Authenticity)	47
3.4.1	Schutzmaßnahmen	48
3.5	Zugriffskontrolle (Access Control)	49
3.5.1	Schutzmaßnahme: <i>Zugriffskontrollsystem (Access Control System)</i> . . .	49
3.6	Nichtabstreitbarkeit, Verbindlichkeit (Non-Repudiation)	50
3.6.1	Schutzmaßnahme: Signaturen und PKI	50
3.7	Verfügbarkeit (Availability)	51
3.7.1	Schutzmaßnahme	52
3.8	Anonymität (Anonymity)	52
4	Verschlüsselungsverfahren	53
4.1	Das Kerckhoffs'sche Prinzip	54
4.2	Mathematische Modellierung von Verschlüsselungsverfahren	55
4.3	Schlüsselaustausch	58
4.4	Angriffsszenarien	59
4.5	Brute-Force Angriffe	60
4.6	Wörterbuchangriffe (dictionary attacks)	62

Historischer Abriss der Kryptographie

1

1.1	Kryptographie und Kryptoanalyse	2
1.2	Klassische Verschlüsselungsverfahren	3
1.3	Das One-Time-Pad: Ein absolut sicheres Verfahren . . .	9
1.4	Das Hill-Verschlüsselungsverfahren: Ein erstes algebraisches Verfahren	12
1.5	Elektromechanische Verschlüsselungsmaschinen	12
1.6	Moderne Verschlüsselungsverfahren	13

1.1 Kryptographie und Kryptoanalyse

Die *Kryptographie* (Lehre von der Geheimschrift (griechisch *kryptos*: verborgen, griechisch *graphein*: schreiben)) fand bis in die 70er Jahre des 20. Jahrhunderts nur im militärischen und diplomatischen Bereichen eine professionelle Anwendung.

Den Verfassern von verschlüsselten Nachrichten standen zu allen Zeiten Kontrahenten gegenüber, die die benutzten kryptographischen Verfahren detailliert analysierten und Schwachstellen der Verfahren mit dem Ziel suchten, verschlüsselte Nachrichten auch ohne weitere *Schlüssel*-Informationen entschlüsseln zu können. Die Entwicklung und der Einsatz von Methoden zum Angriff auf kryptographische Verfahren werden gelegentlich mit dem Begriff *Kryptoanalyse* bezeichnet.

Der vermutlich spektakulärste Erfolg der Kryptoanalyse im letzten Jahrhundert war ein erfolgreicher Angriff auf die verschlüsselten Nachrichten, die während des zweiten Weltkriegs von der deutschen Wehrmacht mit Hilfe der legendären elektromechanischen Chiffriermaschine *Enigma* erstellt wurden. Dieser Erfolg gelang einer Gruppe polnischer und englischer Kryptoanalytiker unter maßgeblicher Beteiligung von *Alan M. Turing*¹.

Tipp: Falls Sie an Details zu diesem erfolgreichen kryptoanalytischen Angriff oder an weiteren spannenden Episoden aus der Geschichte der Kryptographie interessiert sein sollten, sei ihnen das Buch *Geheime Botschaften* von *S. Singh* zur Lektüre wärmstens empfohlen.

Zusammenfassend werden Kryptographie und Kryptoanalyse auch als *Kryptologie* bezeichnet. Kryptologen beschäftigen sich allerdings nicht nur exklusiv mit einem der beiden Teilgebiete der Kryptologie. Der besondere Reiz der Kryptologie ergibt sich vielmehr zu einem großen Teil aus dem Wechselspiel von kryptographischen und kryptoanalytischen Ansätzen.

¹ Alan Mathison Turing (* 1912 London, † 1954 Wilmslow)

1.2 Klassische Verschlüsselungsverfahren

In diesem Abschnitt werden einige kryptographische Verfahren aus der weiter zurückliegenden Vergangenheit beschrieben. Bei dieser Gelegenheit werden bereits einige wichtige Grundbegriffe aus der Kryptographie zur Sprache kommen.

Ziel aller klassischen kryptographischen Verfahren ist die Verschlüsselung einer vertraulichen Nachricht (*Klartextnachricht*) zu einer *Geheimtextnachricht*, die nur von dem adressierten Empfänger entschlüsselt werden kann. Andere Personen, die die Geheimtextnachricht zum Beispiel bei der Übermittlung einsehen können, sollen allein aus der Kenntnis der Geheimtextnachricht keine Rückschlüsse auf die Klartextnachricht ziehen können.

Die klassischen Verfahren sind im Prinzip stets so konstruiert, dass zur Entschlüsselung eines Geheimtextes das jeweilige Verschlüsselungsverfahren rückwärts zu durchlaufen ist. Verschlüsselungs- und Entschlüsselungsverfahren sind einander also sehr ähnlich und die Kenntnis des einen impliziert auch die Kenntnis des anderen Verfahrens. Solche Verfahren werden daher auch *symmetrische Verschlüsselungsverfahren* genannt. Die beiden Grundtechniken bei allen symmetrischen Verschlüsselungsverfahren sind die *Transposition* (Vertauschung) und *Substitution* (Ersetzung) der einzelnen Zeichen einer Nachricht.

Beispiel 1.1: Skytale (Transpositionsverfahren)

Aus dem Altertum ist die Nutzung eines Verschlüsselungsverfahrens durch die Spartaner im 5. Jh. v. Chr. überliefert, welches auf einer Transposition der Zeichen einer Nachricht beruht. Dieses Verfahren nutzt eine so genannte *Skytale* (Holzstab). Zur Verschlüsselung eines Textes wird ein Lederstreifen (Gürtel) um die Skytale gewickelt, und der Text dann in Stabrichtung auf den Streifen geschrieben (je ein Buchstabe auf eine Streifenbahn). Nach Abnahme des Streifens von dem Holzstab sind die Buchstaben in ihrer Anordnung vertauscht. Ein Empfänger der verschlüsselten Nachricht, der das Verfahren kennt, kann nun seinerseits den Streifen auf einen Holzstab gleichen Durchmessers aufrollen, um die Nachricht zu entschlüsseln. ◀

Beispiel 1.2: Caesar-Verfahren (Substitutionsverfahren)

In den von Sueton² verfassten Kaiserbiographien *De vita Caesarum* findet sich die Beschreibung eines von Caesar³ benutzten Verschlüsselungsverfahrens, welches das Substitutionsprinzip nutzt.

Bei diesem *Caesar-Verfahren* wird jeder Buchstabe einer Nachricht durch den drittnächsten Buchstaben des Alphabets (welches zyklisch angeordnet gedacht wird) ersetzt. Aus A wird also D, aus B wird E, usw., bis schließlich Z durch C ersetzt wird. Beispielhaft ist die Verschlüsselung der Klartextnachricht „KLARTEXT“ in Abbildung 1.1 dargestellt.

Natürlich ist der Einsatz dieses Verfahrens nur so lange sinnvoll, wie die Gegner das Verfahren nicht kennen und auch nicht in der Lage sind, aus der Struktur von abgefangenen verschlüsselten Texten das Verfahren und damit die Klartextnachrichten zu ergründen. ◀

² Gaius Suetonius Tranquillus (* um 70, † nach 122)

³ Gaius Julius Caesar (* 100 v. Chr. Rom, † 44 v. Chr. Rom)

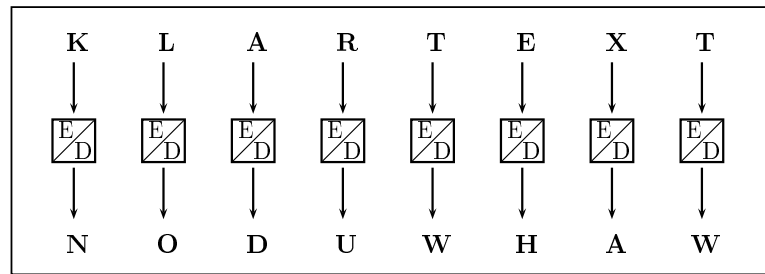


Abb. 1.1 Caesar-Verschlüsselungsverfahren

Frage 1

Entschlüsseln Sie:

YLHO VSDVV PLW GHU NUBSWRJUDSKLH

Setzt ein Verschlüsselungsverfahren zur Wahrung seiner Sicherheit die strikte Geheimhaltung des zugrundeliegenden Algorithmus voraus, so wie das beim Caesar-Verfahren der Fall ist, dann muss dies prinzipiell als ein schwerwiegender Nachteil des Verfahrens angesehen werden: Der Einsatz kann nur innerhalb einer kleinen Nutzergruppe funktionieren, deren Mitglieder alle auf die Geheimhaltung des Verfahrens eingeschworen sein müssen. Gelangt das Wissen über die Funktionsweise eines solchen Verfahrens dennoch an die Öffentlichkeit, sei es durch Unachtsamkeit, Korruption oder Kryptoanalyse, so ist es mit einem Mal vollständig wertlos.

Ein weiterer schwerwiegender Nachteil in der Nutzung eines geheim zu haltenden Verfahrens ist die Tatsache, dass ein solches Verfahren offensichtlich nicht der öffentlichen Kritik, insbesondere der Kritik anerkannter Kryptoanalytiker unterzogen werden kann. Die Sicherheit eines solchen Verfahrens wird daher auch aus diesem Grund immer fragwürdig bleiben. Die Nutzung eines solchen Verfahrens wird daher gelegentlich mit der Phrase „Security by Obscurity“ umschrieben.

Die Lösung der skizzierten Problematik besteht darin, den für das Verschlüsselungsverfahren benutzten Algorithmus parametrisierbar zu machen. Das bedeutet, dass der prinzipiell öffentlich bekannte Verschlüsselungsalgorithmus jeweils zusammen mit einem speziellen (von den jeweiligen Nutzern geheim zu haltenden) Parameterwert verwendet wird. Ein solcher Parameterwert wird auch *Schlüsselwert* oder kurz *Schlüssel* genannt. Die Menge aller für ein Verschlüsselungsverfahren möglichen Schlüssel bildet den sogenannten *Schlüsselraum* $\mathcal{K}$ des Verfahrens.

Bei der Festlegung des Schlüsselraums ist zu beachten, dass dieser hinreichend groß ist, da ein Angreifer sonst einfach alle möglichen Schlüssel ausprobieren könnte, um den passenden zu finden. Angriffe dieser Art werden als *Brute-Force-Angriffe* bezeichnet. Die Größe eines Schlüsselraumes ist natürlich immer in Relation zur Komplexität des benutzten Verschlüsselungsalgorithmus zu bewerten, wobei die *Komplexität* eines Algorithmus formal als die Anzahl der zu seiner Ausführung benötigten (elementaren) Schritte definiert werden kann. Die Komplexität stellt auf diese Weise auch ein Maß für das Laufzeitverhalten eines Algorithmus auf heutigen Rechnern dar.

Das erwähnte Caesar-Verfahren lässt sich in naheliegender Weise parametrisieren, indem zu jeder Zahl n zwischen 1 und 25 eine Verschlüsselung dadurch definiert wird, dass die Buchstaben um n Stellen zyklisch verschoben werden. Der Schlüsselraum hätte hier 25 Elemente

und wäre damit immer noch lächerlich klein. Interessanter wird es jedoch, falls nicht nur zyklische Verschiebungen sondern beliebige Permutationen (d. h. bijektive Selbstabbildungen) der Buchstabenmenge betrachtet werden:

Beispiel 1.3: Monoalphabetische Substitutionsverfahren

Es bezeichne

$$\mathcal{Z} := \{A, B, \dots, Z\}$$

die Menge der 26 Buchstaben (ohne Umlaute) und

$$\mathcal{K} := \Sigma_{\mathcal{Z}} := \{k: \mathcal{Z} \rightarrow \mathcal{Z} \mid k \text{ ist bijektiv}\}$$

den Schlüsselraum des *monoalphabetischen Substitutionsverfahrens* auf $\mathcal{Z}$.

Bei diesem simplen Verfahren gibt ein Schlüssel k bereits direkt an, wie die einzelnen Zeichen einer Klartextnachricht zu verschlüsseln sind. Die Verschlüsselung von $z \in \mathcal{Z}$ mit dem Schlüssel $k \in \mathcal{K}$ ist nämlich:

$$E_k(z) := k(z)$$

Ein Schlüssel k und die Verschlüsselung der Klartextnachricht „KLARTEXT“ mit diesem Schlüssel ist beispielhaft in Abbildung 1.2 dargestellt. ▶

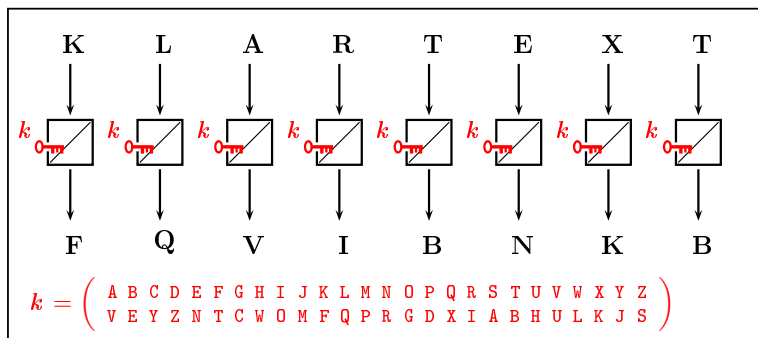


Abb. 1.2 Monoalphabetisches Substitutionsverfahren

Frage 2

Die Bitlänge

$$l_2(a)$$

einer Zahl $a \in \mathbb{N}_0$ ist die Anzahl der Bits, die zur Darstellung der Zahl benötigt wird, d. h. die Anzahl der Stellen in der Binärdarstellung von a . Zeigen Sie, dass für alle $a \in \mathbb{N}$ gilt:

(i) $l_2(a) = \lfloor \log_2(a) \rfloor + 1$

(ii) $l_2(a) = \lceil \log_2(a+1) \rceil$

Frage 3

Wie groß ist der Schlüsselraum bei dem in Beispiel 1.3 beschriebenen monoalphabetischen Substitutionsverfahren?

Geben Sie auch die Bitlänge der Größe des Schlüsselraumes an, d. h., bestimmen Sie:

$$\lfloor \log_2(|\mathcal{K}|) \rfloor + 1$$

Frage 4

Beschreiben Sie die Entschlüsselung beim monoalphabetischen Substitutionsverfahren für eine Geheimtextnachricht.

Frage 5

Schreiben Sie ein kleines Java-Programm zur Erzeugung zufälliger Schlüssel für das monoalphabetische Substitutionsverfahren. Ein Aufruf des Programms soll zur Ausgabe eines Substitutionsschlüssel wie in Abbildung 1.3 führen, wobei jeder Schlüssel mit gleicher Wahrscheinlichkeit $1/|\mathcal{K}|$ auftreten soll.

```
ABCDEFGHIJKLMNPOQRSTUVWXYZ
SYNVADMZFHBPWIXCTGLEKJURQO
```

Abb. 1.3 Ausgabe eines Schlüssels für das monoalphabetische Substitutionsverfahren

Obschon die Größe des Schlüsselraums beim monoalphabetischen Substitutionsverfahren selbst für heutige Maßstäbe recht groß ist, lässt sich eine Nachricht mit Hilfe von *Häufigkeitsanalysen* der in dem Geheimtext vorkommenden Buchstaben und Buchstabenmustern (insbesondere von häufig vorkommenden Buchstabenpaaren) recht leicht wieder dechiffrieren. Diese Möglichkeit wurde zuerst von *al-Kindī*⁴ im 9. Jahrhundert beschrieben.

Frage 6

Eine Anwendung des Substitutionsverfahrens lieferte den Geheimtext:

```
XLH DCUCLSMELIQYAHKEQ HWIHYAYWYACUHTQBNLEBQU GLSY IAH AU
XLH UQWUYQ FLEBEWUXQBY EAUQAU LSH HAKEQB. CIHKECU XAQ
GBCQHHQ XQH HKESWQHHQSBLWDH HQSIHY NWQB EQWYAGQ
DLHHHYLQIQ BQKEY GBCHH AHY, SLQHHY HAKE QAUQ ULKEBAKEY DAY
EASNQ TCU ELQWNAGOQAYHLULSPHQ XQB AU XQD GQEADYQVY
TCBOCDDQUXQU IWKEHYLIQU WUX IWKEHYLIQU DWHYQB BQKEY SQAKEY
JAQXQB XQKEANNBAQBQU. XAQ GBCHHQ BQXWUXLUZ QAUQB HMBLKEQ
DLKEY XAQHQ LULSPHQ DCQGSKE. JQAYQBEAU TAQS HMLHH DAY XQB
OBPMYCGBLMEAQ!
```

Bestimmen Sie mithilfe von CrypTool den Klartext und den benutzten Schlüssel.

⁴ Ya‘qūb Ibn-Ishāq al-Kindī (* um 800 Kufa, †873 Bagdad)

Um Angriffe auf das Substitutionsverfahren abzuwehren, die auf Häufigkeitsanalysen von Buchstaben und speziellen Buchstabenmustern beruhen, wurden von *L. Alberti*⁵, *J. Trithemius*⁶, *G. Belaso*⁷ und *G. Porta*⁸ *polyalphabetische Substitutionsverfahren* entwickelt, bei denen der Schlüssel für die Verschlüsselung eines einzelnen Zeichens periodisch gewechselt wird. Der Schlüsselwechsel wird hierbei meist selbst wiederum durch ein Schlüsselwort kodiert. Dieses Verfahren wurde 1585 von *B. de Vigenère*⁹ in seinem *Traicté des Chiffres* diskutiert und wird meist als *Vigenère-Verfahren* bezeichnet.

Beispiel 1.4: Vigenère-Verfahren (Polyalphabetische Substitution)

Es bezeichne

$$\mathcal{Z} := \{A, B, \dots, Z\}$$

die Menge der 26 Buchstaben (ohne Umlaute) und

$$\mathcal{M} = \mathcal{Z}^{>0} := \bigcup_{n \in \mathbb{N}} \mathcal{Z}^n = \{(m_1, m_2, \dots, m_n) \mid m_i \in \mathcal{Z}, n \in \mathbb{N}\}$$

die Menge aller endlichen Folgen mit Zeichen aus $\mathcal{Z}$.

Der Schlüsselraum des *Vigenère-Verfahrens* ist durch

$$\mathcal{K} := \Sigma_{\mathcal{Z}} \times \mathcal{M}$$

definiert. Ein Schlüssel $k = (f, w) \in \mathcal{K}$ besteht also neben einer Permutation $f \in \Sigma_{\mathcal{Z}}$ (wie beim monoalphabetischen Substitutionsverfahren) zusätzlich noch aus einem Schlüsselwort w .

Jeder Buchstabe einer Klartextnachricht wird wie beim monoalphabetischen Substitutionsverfahren durch die Anwendung einer Permutation substituiert. Allerdings wird nun nicht für jedes Zeichen dieselbe Permutation verwendet. Vielmehr wird die Permutation f des Schlüssels entsprechend der Buchstaben des Schlüsselworts zyklisch modifiziert, bevor die so modifizierte Permutation zur Verschlüsselung eines Zeichens zur Anwendung kommt. Zur Modifikation von f durch einen Buchstaben werden die Bilder von f zyklisch verschoben, bis der besagte Buchstabe das Bild des Buchstabens A ist, d. h. an erster Stelle steht.

Für den Schlüssel, der aus der in Abbildung 1.2 dargestellten Permutation und dem Schlüsselwort SKAT besteht, ist die Verschlüsselung der Klartextnachricht „Klartext“ beispielhaft in Abbildung 1.4 dargestellt. ◀

⁵ Leon Battista Alberti (* 1404 Genua, † 1472 Rom)

⁶ Johannes Trithemius (* 1462 Trittenheim, † 1516 Würzburg)

⁷ Giovanni Battista Belaso (* 1505 Brescia, † zwischen 1568 und 1581 Rom (?))

⁸ Giovanni Battista della Porta (* 1535 Vico Equense, † 1615 Neapel)

⁹ Blaise de Vigenère (* 1523 Saint-Pourçain-sur-Sioule, † 1596 Paris)

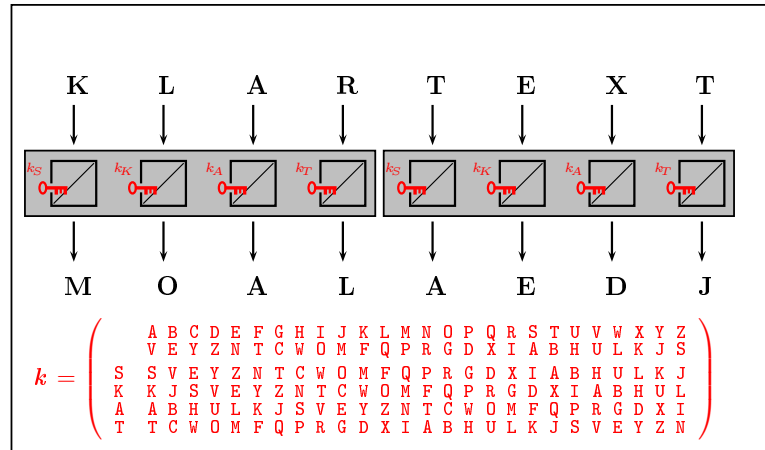


Abb. 1.4 Polyalphabetisches Verschlüsselungsverfahren (Vigenère-Verfahren)

Das Vigenère-Verfahren ist ein sogenanntes *Blockverschlüsselungsverfahren*, da jeweils ein gleichlanger Block von Zeichen zusammengefasst wird und dem gleichen Verschlüsselungsalgorithmus unterworfen wird. (In dem in Abbildung 1.4 dargestellten Beispiel sind es vier Zeichen.)

Das Vigenère-Verfahren ist allerdings bei hinreichend langen Texten ebenfalls mit Hilfe statistischer Methoden recht leicht zu brechen. Hierzu sind die Teilfolgen der Zeichen im Geheimtext mit gleichem Abstand n für verschiedene natürliche Zahlen $n = 1, 2, \dots$ zu bilden und die Verteilung der Zeichen in diesen Folgen mit der durchschnittlichen Verteilung der Zeichen in Texten der Sprache des Klartextes zu vergleichen. Lässt sich für die verschiedenen Teilfolgen zu einem möglichst kleinen n jeweils eine gute Übereinstimmung feststellen, so ist dieses n mit hoher Wahrscheinlichkeit die Blocklänge. Die einzelnen Schlüssel, die innerhalb eines Blockes angewandt werden müssen, ergeben sich mit Hilfe des bereits für die entsprechende Folge durchgeführten Vergleichs der Anzahl vorkommender Zeichen.

Die Blocklänge im Vigenère-Verfahren kann übrigens noch geschickter bestimmt werden, indem der Abstand von wiederholt auftretenden Zeichenfolgen (von drei oder mehr Zeichen) ermittelt wird. Solche Zeichenfolgen werden nämlich in der Regel von jeweils identischen Zeichenfolgen des Klartextes stammen, die unter der gleichen Teilschlüsselfolge verschlüsselt wurden. Das diese Folgen unter der gleichen Teilschlüsselfolge verschlüsselt wurden, bedeutet aber, dass der Abstand ihrer Positionen innerhalb des Textes ein Vielfaches der Blocklänge sein muss. Eine Berechnung des größten gemeinsamen Teilers all solcher Abstände liefert häufig bereits die Blocklänge (oder ein kleines Vielfaches der Blocklänge). Diese Vorgehensweise wurde zuerst von *F. Kasiski*¹⁰ im Jahre 1863 veröffentlicht und wird daher als *Kasiski-Test* bezeichnet. Allerdings wurde diese Vorgehensweise auch bereits etwa 10 Jahre zuvor von *C. Babbage*¹¹ gefunden.

Nach der Bestimmung der Blocklänge bleibt natürlich immer noch die Aufgabe zu lösen, die Teilfolgen der Zeichen vom Abstand der ermittelten Blocklänge zu bilden und die Häufigkeiten der auftretenden Zeichen mit der durchschnittlichen Verteilung in einem Text der Sprache des Klartextes zu vergleichen.

¹⁰ Friedrich Wilhelm Kasiski (* 1805 Schlochau, † 1881 Neustettin)

¹¹ Charles Babbage (* 1791 Walworth, † 1871 London)

1.3 Das One-Time-Pad: Ein absolut sicheres Verfahren

Wird jedes Zeichen einer Klartextnachricht unabhängig von den anderen Zeichen mit jeweils gleicher Wahrscheinlichkeit durch ein anderes Zeichen substituiert, so lässt der entstehende Geheimtext keinerlei Rückschlüsse auf den Inhalt der Klartextnachricht zu. (Natürlich bleibt die Länge der Nachricht unverändert. Falls sich also aus der Länge einer Nachricht Informationen ableiten lassen, sind Klartextnachrichten vor einer Verschlüsselung auf eine einheitliche Länge zu bringen.)

Eine gleichwahrscheinliche Verteilung bei der Substitution lässt sich sehr einfach durch die Anwendung eines je Zeichen zufällig gewählten verallgemeinerten Caesar-Verfahrens erreichen. Zur Beschreibung des Verfahrens soll zunächst das (verallgemeinerte) Caesar-Verfahren in eine mathematische Darstellungsform gebracht werden:

Werden die Buchstaben, d. h. die Elemente der Menge

$$\mathcal{Z} = \{A, B, C, \dots, Z\}$$

mit den Elementen der Menge

$$\mathbb{Z}_{26} = \{0, 1, 2, \dots, 25\}$$

identifiziert, so lässt sich die Caesar-Verschlüsselung eines Buchstabens mithilfe einer Addition mit 3 modulo 26 darstellen:

$$E(m) = (m + 3) \bmod 26 \quad (1.1)$$

Bei dem verallgemeinerten Caesar-Verfahren wird anstelle der Zahl 3 eine andere Zahl aus $\mathbb{Z}_{26}$ zur Addition benutzt.

Für jede natürliche Zahl $n \geq 2$ wird die Menge $\mathbb{Z}_n$ mit der Addition modulo n sowie einer entsprechenden Multiplikation modulo n zu einem kommutativen Ring. Diese endlichen Zahlbereiche spielen in der Kryptographie eine herausragende Rolle und werden im nächsten Kapitel daher ausführlich behandelt. Für den Moment benötigen wir nur die Eigenschaft, dass Zahlen aus $\mathbb{Z}_n$ innerhalb von $\mathbb{Z}_n$ addiert werden können und Inverse bezüglich der Addition besitzen, sodass auch eine Subtraktion innerhalb von $\mathbb{Z}_n$ definiert ist.

Wichtig Bei einer Anwendung einer arithmetischen Operation (Addition, Subtraktion, Multiplikation und (falls definiert) Division) auf Elemente aus einem gegebenen Zahlbereich $\mathbb{Z}_n$ soll immer die modulare Operation verstanden werden.

Beispielsweise schreibt sich Gleichung (1.1) für $m \in \mathbb{Z}_n$ und $3 \in \mathbb{Z}_n$ einfach als:

$$E(m) = m + 3 \quad (1.2)$$



Beispiel 1.5: One-Time-Pad-Verfahren

Wir gehen von folgenden Annahmen aus:

Klar- und Geheimtextnachrichten werden als Folgen von Zeichen der Zeichenmenge

$$\mathcal{Z} = \mathbb{Z}_n \quad (n \in \mathbb{N})$$

aufgefasst:

$$\mathcal{M} = \mathcal{Z}^{>0} := \bigcup_{l \in \mathbb{N}} \mathbb{Z}_n^l = \{(m_1, m_2, \dots, m_l) \mid m_i \in \mathbb{Z}_n, l \in \mathbb{N}\}$$

Zur Verschlüsselung einer Klartextnachricht

$$m = (m_1, m_2, \dots, m_l) \in \mathbb{Z}_n^l$$

kommt als Schlüssel ein zufällig gewähltes *One Time Pad*

$$k = (k_1, k_2, \dots, k_l) \in \mathbb{Z}_n^l$$

zum Einsatz:

$$E_k(m) := m + k := (m_1 + k_1, m_2 + k_2, \dots, m_l + k_l)$$

Kommentar Bei einer Identifizierung der Buchstaben $A, B, \dots, Z$ mit den Zahlen aus $\mathbb{Z}_{26}$ und bei einem gegebenen One-Time-Pad $(k_1, k_2, \dots, k_l) \in \mathbb{Z}_{26}^l$ wird jedes Zeichen m_i mit dem verallgemeinerten Caesar-Verfahren durch Addition mit k_i in $\mathbb{Z}_{26}$ verschlüsselt. ◀

Kommentar Das One-Time-Pad-Verfahren wurde bereits im Jahr 1882 von F. Miller¹² in seinem *Codebook (Telegraphic Code to Insure Privacy and Secrecy in the Transmission of Telegrams)* beschrieben. Miller's Arbeit fand allerdings keine weite Verbreitung und gut 35 Jahre später wurde das Verfahren von G. Vernam¹³ nochmals erfunden und zum Patent angemeldet. Es wird daher seither manchmal auch als *Vernam-Verfahren* bezeichnet. ▶

Das One-Time-Pad-Verfahren besitzt die folgende bemerkenswerte Eigenschaft:

Satz 1.1 (Perfekte Sicherheit des One-Time-Pad-Verfahrens)

Werden die Zeichen k_i des Schlüssels mit einem perfekten Zufallszahlengenerator erzeugt, bei dem jede Zahl aus $\mathbb{Z}_n$ mit der gleichen Wahrscheinlichkeit $\frac{1}{n}$ auftritt, so ermöglicht eine Kenntnis der Geheimtextnachricht

$$c = (c_1, c_2, \dots, c_l) = ((m_1 + k_1) \bmod n, (m_2 + k_2) \bmod n, \dots, (m_l + k_l) \bmod n)$$

keinerlei Rückschlüsse auf die Klartextnachricht $(m_1, m_2, \dots, m_l)$.

Beweis Zu jeder möglichen Klartextnachricht $\tilde{m} = (\tilde{m}_1, \tilde{m}_2, \dots, \tilde{m}_l)$ gibt es genau einen Schlüssel $\tilde{k} = (\tilde{k}_1, \tilde{k}_2, \dots, \tilde{k}_l)$ mit:

$$\tilde{m} + \tilde{k} = c$$

Da jeder Schlüssel mit der gleichen Wahrscheinlichkeit $(\frac{1}{n})^l$ in Betracht zu ziehen ist, ist auch jede mögliche Klartextnachricht mit der gleichen Wahrscheinlichkeit die richtige Klartextnachricht. ■

Wichtig Ein One-Time-Pad darf, so wie es der Name zum Ausdruck bringt, nur ein einziges Mal für eine Verschlüsselung verwendet werden. Andernfalls ergibt sich aus zwei Geheimtextnachrichten, die mit dem gleichen One-Time-Pad k verschlüsselt wurden

$$c = m + k, \quad \tilde{c} = \tilde{m} + k$$

durch Berechnung der Differenz

$$c - \tilde{c} = m + k - (\tilde{m} + k) = m - \tilde{m}$$

¹² Frank Miller (* 1842 Milwaukee, † 1925)

¹³ Gilbert Sandford Vernam (* 1890 New York, † 1960)

auch die Differenz der Klartextnachrichten m und $\tilde{m}$. Sind Teile einer der beiden Klartextnachrichten bekannt, so können die entsprechenden Teile der anderen Nachricht sofort bestimmt werden. Darüber hinaus können in der Regel bereits durch die Differenzen der Klartextnachrichten Erkenntnisse über die beiden Nachrichten gewonnen werden (mittels Häufigkeitsanalysen, etc.). ◀

Kommentar Trotz der bewiesenen perfekten Sicherheit des One-Time-Pad-Verfahrens wird es selten genutzt. Das Problem bei praktischen Anwendungen ist, das vor einer Verschlüsselung ein Schlüssel auszutauschen ist, der mindestens so lang ist, wie die zu einem späteren Zeitpunkt zu verschlüsselnde Klartextnachricht. ◀

1.4 Das Hill-Verschlüsselungsverfahren: Ein erstes algebraisches Verfahren

Als letztes historisches Beispiel soll das von *L. Hill*¹⁴ im Jahre 1929 eingeführte und nach ihm benannte *Hill-Verschlüsselungsverfahren* behandelt werden, da es ein Verfahren ist, welches eine etwas interessantere mathematische Operation nutzt.

Beispiel 1.6: Hill-Verfahren

Wir gehen von folgenden Annahmen aus, wobei $n, k \in \mathbb{N}$ fest vorgegeben sind:

Klar- und Geheimtextnachrichten sind Folgen von Blöcken (Vektoren) der Länge k , deren Komponenten Zeichen aus $\mathbb{Z}_n$ sind. Die Blöcke sind also Elemente der Menge:

$$\mathcal{B} = \mathbb{Z}_n^k$$

Zur Verschlüsselung von Klartextblöcken $\mathbf{b} \in \mathcal{B}$ kommt als Schlüssel eine zufällig gewählte invertierbare (k, k) -Matrix $\mathbf{K}$ zum Einsatz:

$$E_{\mathbf{K}}(\mathbf{b}) := \mathbf{K} \cdot \mathbf{b}$$

Zur Entschlüsselung eines Geheimtextblocks $\mathbf{c}$ wird die zu $\mathbf{K}$ inverse Matrix benötigt:

$$D_{\mathbf{K}}(\mathbf{c}) := \mathbf{K}^{-1} \cdot \mathbf{c}$$

Wie lässt sich von einer zufällig gewählten (k, k) -Matrix $\mathbf{K}$ mit Koeffizienten aus $\mathbb{Z}_n$ feststellen, ob sie invertierbar ist? Und wie lässt sich gegebenenfalls die Inverse $\mathbf{K}^{-1}$ berechnen? Diese Fragen werden im nächsten Kapitel in Abschnitt 2.3 geklärt!

Frage 7

Verschlüsseln Sie den Klartext DATENSICHERHEIT mit Hilfe des Hill-Verfahrens zur Matrix

$$\begin{pmatrix} 10 & 12 & 21 \\ 3 & 21 & 11 \\ 23 & 20 & 21 \end{pmatrix}$$

über $\mathbb{Z}_{26}$, wobei die Buchstaben A, B, ..., Z mit $0, 1, \dots, 25 \in \mathbb{Z}_{26}$ identifiziert werden.

1.5 Elektromechanische Verschlüsselungsmaschinen

Die erste Hälfte des letzten Jahrhunderts war die Zeit der elektromechanischen Chiffriermaschinen, deren bekanntestes Beispiel wohl die deutsche Enigma ist. Diese Maschinen sind so konstruiert, dass über eine Reihe von Rotorscheiben, deren Stellungen sich nach der Verschlüsselung eines Zeichens jeweils verändern, für jedes Zeichen eine neue Substitution bewirkt wird.

Durch die Möglichkeiten moderner Computer sind den Kryptographen natürlich wesentlich mächtigere Hilfsmittel in die Hand gegeben worden, den Kryptoanalytikern allerdings auch.

¹⁴ Lester Sanders Hill (* 1891 New York, † 1961 New York)

1.6 Moderne Verschlüsselungsverfahren

Mit der breiten Einführung der elektronischen Datenverarbeitung für kommerzielle Anwendungen wurde in diesem Bereich die Notwendigkeit für den Einsatz von standardisierten kryptographischen Verfahren erkannt. Der für viele Jahrzehnte wichtigste Standard war der 1976 veröffentlichte *Data Encryption Standard (DES)*, der selbst heutzutage noch manchmal genutzt wird (meist in einer stärkeren Variante als *Tripel-DES*). Durch die immense Bedeutung der modernen Informationstechnologien und der mit ihrem Einsatz verbundenen Sicherheitsfragen, z.B. bei der Übertragung von sensiblen persönlichen oder kommerziellen Daten über ungeschützte Netze (Internet), wurden seit den 70er Jahren des letzten Jahrhunderts kryptographische Algorithmen zunehmend auch von Spezialisten an Universitäten und industriellen Forschungseinrichtungen entwickelt und standardisiert. Als Beispiel für ein offenes Verfahren zur Standardisierung eines kryptographischen Algorithmus kann das Auswahlverfahren für die Festlegung des *Advanced Encryption Standards (AES)* zur Ablösung des DES-Algorithmus genannt werden, welches vom *National Institute of Standards and Technology (NIST)* in den Jahren 1997-2000 in drei Runden durchgeführt wurde und mit der Nominierung des *Rijndael*-Algorithmus zum Abschluss kam.

Mitte der 1970er Jahre wurden von *R. Merkle*¹⁵, *W. Diffie*¹⁶ und *M. Hellman*¹⁷ erste Ideen zu der seinerzeit revolutionären *Public Key Cryptography (asymmetrische Kryptographie)* entwickelt und publiziert. Insbesondere ist in diesem Zusammenhang die von W. Diffie und M. Hellman im Jahre 1976 publizierte Arbeit *New directions in cryptography* zu nennen, die eine Beschreibung des nach den Autoren benannten *Diffie-Hellman-Verfahrens* (kurz: *DH-Verfahren*) zum Schlüsselaustausch über unsichere Kanäle beinhaltet. Ein weiteres asymmetrisches Verfahren wurde 1977 von *R. Rivest*¹⁸, *A. Shamir*¹⁹ und *L. Adleman*²⁰ entwickelt. Dieses nach den Initialen der Erfinder benannte *RSA-Verfahren* lässt sowohl als asymmetrisches Verschlüsselungsverfahren als auch als Signaturverfahren einsetzen.

Ohne die Verfahren der asymmetrischen Kryptographie wäre die Realisierung sicherer Kommunikationsverbindungen, so wie sie heutzutage in den vielfältigsten Anwendungen des elektronischen Datenaustauschs benötigt werden, nicht denkbar. Beispielsweise werden die Möglichkeiten zu einem sicheren *Schlüsselaustausch* und zur Nutzung elektronischer *Signaturen* für eine *Authentifikation* von Kommunikationspartnern bei einem Verbindungsaufbau (*Handshake*) zu einem Webserver mit dem HTTPS-Protokoll (HTTP über TLS) benötigt.

Bemerkenswert ist die Tatsache, dass die oben genannten DH- und RSA-Verfahren bis zum heutigen Tag quasi exklusiv als asymmetrische Verfahren zur Anwendung kommen. (Bezüglich des DH-Verfahrens wurde lediglich eine Variante eingeführt, bei der anstelle eines Zahlbereichs $\mathbb{Z}_n$ mit einer großen Zahl n die Punktmenge einer über einem endlichen Körper definierten elliptischen Kurve tritt.)

¹⁵ Ralph C. Merkle (* 1952 Berkeley)

¹⁶ Whitfield Diffie (* 1944 Washington, D.C.)

¹⁷ Martin Edward Hellman (* 1945 New York)

¹⁸ Ronald Linn Rivest (* 1947 Schenectady)

¹⁹ Adi Shamir (* 1952 Tel-Aviv)

²⁰ Leonard Max Adleman (* 1945 San Francisco)

Modulare Arithmetik

2

2.1	Der Ring $\mathbb{Z}_n$	16
2.2	Inverse Elemente in $\mathbb{Z}_n$ bezüglich der Multiplikation . . .	21
2.3	Matrizen über $\mathbb{Z}_n$	24
2.4	Der erweiterte Euklid'sche Algorithmus	26
2.5	Berechnung inverser Elemente in $\mathbb{Z}_n$	34
2.6	Mathematischer Exkurs: <i>Euler'sche φ-Funktion</i>	36
2.7	Primfaktorzerlegung natürlicher Zahlen	38

2.1 Der Ring $\mathbb{Z}_n$

Der Zahlbereich $\mathbb{Z}_n$

Für $n \in \mathbb{N}$ mit $n \geq 2$ wird die Menge $\mathbb{Z}_n$ definiert durch:

$$\mathbb{Z}_n := \{0, 1, \dots, n-1\}$$

Auf dem Zahlbereich $\mathbb{Z}_n$ lässt sich eine Addition und Multiplikation definieren. Hierzu wird die Division ganzer Zahlen mit Rest benötigt, für die die folgenden Bezeichnungen festgelegt werden:

Division mit Rest

Sind $a, b \in \mathbb{Z}, b \neq 0$, so gibt es eindeutig bestimmte Elemente $q, r \in \mathbb{Z}, 0 \leq r < |b|$ mit:

$$a = b \cdot q + r \quad (2.1)$$

Die Zahl q ist der *Quotient* und r der *Rest*, der sich bei Division von a durch b ergibt. Für den Quotienten kann

$$a /_{\mathbb{Z}} b := q \quad (2.2)$$

geschrieben werden, um Verwechslungen mit dem Ergebnis der Division in $\mathbb{Q}$ auszuschließen. Für den Rest wird

$$a \bmod b := r \quad (2.3)$$

geschrieben.

Gleichung (2.1) schreibt sich also mit diesen Bezeichnungen wie folgt:

$$a = b \cdot (a /_{\mathbb{Z}} b) + (a \bmod b) \quad (2.4)$$

Frage 8

Berechnen Sie:

- (i) $3885 /_{\mathbb{Z}} 700$ und $3885 \bmod 700$
- (ii) $3885 /_{\mathbb{Z}} (-700)$ und $3885 \bmod (-700)$
- (iii) $-3885 /_{\mathbb{Z}} 700$ und $-3885 \bmod 700$
- (iv) $-3885 /_{\mathbb{Z}} (-700)$ und $-3885 \bmod (-700)$

Frage 9

Für die im Rahmen einer Programmiersprache darstellbaren ganzen Zahlen sind in der Regel auch Divisionsoperatoren definiert. Entsprechen diese Operatoren den durch (2.2) und (2.3) definierten Verknüpfungen auf $\mathbb{Z}$?

Konkret: Entsprechen in der Programmiersprache Java die Operatoren `/` und `%` den Verknüpfungen $/_{\mathbb{Z}}$ bzw. $\bmod$? Überprüfen Sie dies mit den Beispielen aus Frage 8.

Durch *Reduktion modulo n* des Ergebnisses einer Addition oder Multiplikation von zwei Elementen aus $\mathbb{Z}_n$ wird auf der Menge $\mathbb{Z}_n$ eine Addition $+\mathbb{Z}_n$ und eine Multiplikation $\cdot\mathbb{Z}_n$ definiert:

Addition und Multiplikation in $\mathbb{Z}_n$

Für $a, b \in \mathbb{Z}_n$ sind definiert:

$$a +_{\mathbb{Z}_n} b := (a + b) \bmod n \quad (2.5)$$

$$a \cdot_{\mathbb{Z}_n} b := (a \cdot b) \bmod n \quad (2.6)$$

Inverse bezüglich der Addition und Subtraktion in $\mathbb{Z}_n$

Bezüglich der Addition gibt es in $\mathbb{Z}_n$ das *Nullelement* 0 und jedes $a \in \mathbb{Z}_n$ besitzt ein (additives) Inverses, nämlich:

$$-a := \begin{cases} 0 & \text{für } a = 0 \\ n - a & \text{sonst} \end{cases} \quad (2.7)$$

Die Subtraktion wird für Elemente $a, b \in \mathbb{Z}_n$ mithilfe der Addition und Inversenbildung (bezüglich der Addition) erklärt:

$$a -_{\mathbb{Z}_n} b := a +_{\mathbb{Z}_n} (-b) \bmod n \quad (2.8)$$

Wichtig Ist aus dem Zusammenhang klar, dass die Summe, die Differenz oder das Produkt von zwei Elementen aus $\mathbb{Z}_n$ innerhalb von $\mathbb{Z}_n$ gebildet werden soll, so schreiben wir auch nur $+$, $-$ bzw. $\cdot$ an Stelle von $+\mathbb{Z}_n$, $-\mathbb{Z}_n$ bzw. $\cdot\mathbb{Z}_n$. ◀

Beispiel 2.1

In $\mathbb{Z}_6$ gilt:

$$3 + 4 = 1$$

$$3 \cdot 3 = 3$$

$$2 \cdot 3 = 0$$

$$3 - 5 = 3 + 1 = 4$$

Sollen mehrere Rechenschritte in $\mathbb{Z}_n$ hintereinander ausgeführt werden, so können die entsprechenden Rechnungen auch zunächst in $\mathbb{Z}$ durchgeführt werden. Eine anschließende Reduktion des so erhaltenen Ergebnisses modulo n ergibt dann das gleiche Ergebnis in $\mathbb{Z}_n$. Dies wird etwas später formal mit Satz 2.4 begründet. Zunächst sollen jedoch noch einige weitere Bezeichnungen eingeführt und grundlegende Zusammenhänge festgehalten werden.

Teiler, Vielfache

Per Definition *teilt* $b \in \mathbb{Z}$ die Zahl $a \in \mathbb{Z}$, falls es ein $q \in \mathbb{Z}$ gibt mit:

$$a = b \cdot q$$

In diesem Fall heißt b ein *Teiler* von a , a ein *Vielfaches* von b und es wird hierfür

$$b \mid a$$

geschrieben. Für den Fall, dass b die Zahl a nicht teilt, wird

$$b \nmid a$$

geschrieben.

Satz 2.1

- (i) Es gilt $a \mid 0$ für alle $a \in \mathbb{Z}$.
- (ii) Gilt $a \mid b$, so gilt auch $a \mid (-b)$.
- (iii) Gilt $a \mid b$ und $a \mid c$, so gilt auch $a \mid (b+c)$.

Kongruenzen

Sind $a, b \in \mathbb{Z}$ so heißen a und b *kongruent modulo* n , falls $n \in \mathbb{N}$ die Differenz $a - b$ teilt. Hierfür wird auch

$$a \equiv b \pmod{n}$$

geschrieben.

Aus der Definition folgt direkt:

Für alle $a, b \in \mathbb{Z}$ und $n \in \mathbb{N}$ gilt:

- (i) $a \equiv 0 \pmod{n} \iff n \mid a$
- (ii) $a \equiv b \pmod{n} \iff (a-b) \equiv 0 \pmod{n}$

Satz 2.2

Für jede natürliche Zahl n definiert die Relation der *Kongruenz modulo* n eine *Äquivalenzrelation* auf $\mathbb{Z}$, d.h. für alle $a, b, c \in \mathbb{Z}$ gilt:

- (i) *Reflexivität:* $a \equiv a \pmod{n}$
- (ii) *Symmetrie:* $a \equiv b \pmod{n} \implies b \equiv a \pmod{n}$
- (iii) *Transitivität:* $\left. \begin{array}{l} a \equiv b \pmod{n} \\ b \equiv c \pmod{n} \end{array} \right\} \implies a \equiv c \pmod{n}$

Beweis

- (i) n teilt $0 = a - a$.
- (ii) Teilt n die Differenz $a - b$, so teilt n auch $-(a - b) = b - a$.
- (iii) Teilt n die Differenzen $a - b$ und $b - c$, so teilt n auch $(a - b) + (b - c) = a - c$.

■

Satz 2.3

Es gilt:

$$a \bmod n = b \bmod n \iff a \equiv b \pmod{n}$$

Beweis Es seien

$$a = v_1 \cdot n + r_1 \quad \text{und} \quad b = v_2 \cdot n + r_2$$

mit:

$$v_1, v_2, r_1, r_2 \in \mathbb{Z} \quad \text{und} \quad 0 \leq r_1, r_2 < n$$

Dann gilt:

$$\begin{aligned}
 a \bmod n = b \bmod n &\iff r_1 = r_2 \\
 &\iff n \mid (r_1 - r_2) \quad (\text{wegen } 0 \leq r_1, r_2 < n) \\
 &\iff n \mid (r_1 - r_2) + (v_1 - v_2) \cdot n \\
 &\iff n \mid (a - b) \\
 &\iff a \equiv b \pmod{n}
 \end{aligned}$$

■

Satz 2.4Sind $a, a', b, b' \in \mathbb{Z}$ mit

$$a \equiv a' \pmod{n} \quad \text{und} \quad b \equiv b' \pmod{n}$$

so gilt:

$$a + b \equiv a' + b' \pmod{n} \tag{2.9}$$

$$a \cdot b \equiv a' \cdot b' \pmod{n} \tag{2.10}$$

Insbesondere gilt für beliebige $a, b \in \mathbb{Z}$:

$$((a \bmod n) + (b \bmod n)) \bmod n = (a + b) \bmod n$$

$$((a \bmod n) \cdot (b \bmod n)) \bmod n = (a \cdot b) \bmod n$$

Beweis Wegen

$$a \equiv a' \pmod{n} \quad \text{und} \quad b \equiv b' \pmod{n}$$

gibt es

$$v_1, v'_1, r_1, v_2, v'_2, r_2 \in \mathbb{Z} \quad \text{und} \quad 0 \leq r_1, r_2 < n$$

mit:

$$\begin{aligned} a &= v_1 \cdot n + r_1, & a' &= v'_1 \cdot n + r_1, \\ b &= v_2 \cdot n + r_2, & b' &= v'_2 \cdot n + r_2 \end{aligned}$$

Da

$$\begin{aligned} (a+b) - (a'+b') &= (a-a') + (b-b') \\ &= (v_1 - v'_1) \cdot n + (v_2 - v'_2) \cdot n \\ &= (v_1 - v'_1 + v_2 - v'_2) \cdot n \end{aligned}$$

durch n geteilt wird gilt (2.9).

Ebenso wird

$$\begin{aligned} a \cdot b - a' \cdot b' &= a \cdot b - a' \cdot b + a' \cdot b - a' \cdot b' \\ &= (a - a') \cdot b + a' \cdot (b - b') \\ &= (v_1 - v'_1) \cdot n \cdot b + a' \cdot (v_2 - v'_2) \cdot n \\ &= ((v_1 - v'_1) \cdot b + a' \cdot (v_2 - v'_2)) \cdot n \end{aligned}$$

durch n geteilt und es gilt daher auch (2.10). ■

Wird modulo einer natürlichen Zahl n gerechnet, so sind also Zahlen, die sich um ein Vielfaches von n unterscheiden, gewissermaßen gleichwertig, und jede ganze Zahl kann durch genau eine Zahl aus der Menge $\mathbb{Z}_n$ vertreten werden.

Wichtig Die in $\mathbb{Z}$ gültigen Assoziativ-, Kommutativ- und Distributivgesetze übertragen sich mit Hilfe von Satz 2.4 sofort auf den Zahlbereich $\mathbb{Z}_n$. $\mathbb{Z}_n$ bildet für $n \geq 2$ daher mit den in (2.5) und (2.6) definierten Verknüpfungen $+_{\mathbb{Z}_n}$ und $\cdot_{\mathbb{Z}_n}$ einen *kommutativen Ring* mit *Einselement* 1.

Im Folgenden wird von den in $\mathbb{Z}_n$ geltenden Rechenregeln und Gesetzen vielfach Gebrauch gemacht. ◀

Frage 10

Berechnen Sie 33^{37} in $\mathbb{Z}_{43}$.

2.2 Inverse Elemente in $\mathbb{Z}_n$ bezüglich der Multiplikation

Ähnlich wie in $\mathbb{Z}$ kann auch in $\mathbb{Z}_n$ die Division für zwei Elemente aus $\mathbb{Z}_n \setminus \{0\}$ im Allgemeinen nicht vernünftig definiert werden. Um alle Zahlen aus $\mathbb{Z}_n$ durch eine gegebene Zahl $a \in \mathbb{Z}_n$ teilen zu können, muss a ein *inverses Element* (bezüglich der Multiplikation) besitzen, d.h. es muss ein Element $b \in \mathbb{Z}_n$ geben, für das

$$a \cdot b = 1$$

in $\mathbb{Z}_n$ gilt. (b ist dann das Ergebnis einer Division von 1 durch a .)

Besitzt a ein inverses Element, so ist dieses eindeutig bestimmt. Sind nämlich b und c Elemente in $\mathbb{Z}_n$ mit

$$a \cdot b = a \cdot c = 1$$

in $\mathbb{Z}_n$, so gilt in $\mathbb{Z}_n$ auch:

$$b = b \cdot 1 = b \cdot (a \cdot c) = (b \cdot a) \cdot c = 1 \cdot c = c$$

Definition

Ein Element $a \in \mathbb{Z}_n$ heißt *invertierbar*, falls es ein Element $b \in \mathbb{Z}_n$ mit $a \cdot b = 1$ gibt. Das Element b wird als (*multiplikativ*) *inverses Element* zu a oder auch nur kurz als das *Inverse* zu a bezeichnet. Für dieses Element wird

$$a^{-1} := b$$

geschrieben.

Die Menge aller Elemente aus $\mathbb{Z}_n$, die ein inverses Element besitzen, werden in der Menge $\mathbb{Z}_n^*$ zusammengefasst:

Definition

Die *Einheitengruppe* von $\mathbb{Z}_n$ ist die Menge:

$$\mathbb{Z}_n^* = \{a \in \mathbb{Z}_n \mid a \cdot b = 1 \text{ für ein } b \in \mathbb{Z}_n\}$$

Beispiel 2.2

In $\mathbb{Z}_6$ gilt $5^{-1} = 5$. Die Elemente 2, 3, 4 besitzen in $\mathbb{Z}_6$ keine Inversen. Es gilt also:

$$\mathbb{Z}_6^* = \{1, 5\}$$

Beispiel 2.3

In $\mathbb{Z}_{10}$ sind 1, 3, 7 und 9 invertierbar, da gilt:

$$3 \cdot 7 = 9^2 = 1$$

Da die anderen Elemente aus $\mathbb{Z}_{10}$ nicht invertierbar sind folgt wegen

$$2 \cdot 5 = 4 \cdot 5 = 6 \cdot 5 = 8 \cdot 5 = 0$$

mit dem folgenden Satz 2.5 (iii). Es gilt also:

$$\mathbb{Z}_{10}^* = \{1, 3, 7, 9\}$$

Satz 2.5

- (i) Die Einheitengruppe $\mathbb{Z}_n^*$ ist bezüglich der Produktbildung abgeschlossen, d. h.:

$$a, b \in \mathbb{Z}_n^* \implies a \cdot b \in \mathbb{Z}_n^*$$

- (ii) Kürzungsregel:

$$a \cdot b = a \cdot c, \quad a \in \mathbb{Z}_n^* \implies b = c$$

- (iii) Sind $a, b \in \mathbb{Z}_n \setminus \{0\}$ und ist $a \cdot b = 0$ in $\mathbb{Z}_n$, so folgt $a \notin \mathbb{Z}_n^*$ und $b \notin \mathbb{Z}_n^*$.

- (iv) Für $a \in \mathbb{Z}_n$ mit $\text{ggT}(a, n) \neq 1$ gilt $a \notin \mathbb{Z}_n^*$.

Beweis

- (i) Seien $a, b \in \mathbb{Z}_n^*$. Dann besitzen sie Inverse a^{-1} bzw. b^{-1} . Für $a^{-1} \cdot b^{-1}$ gilt:

$$(a \cdot b) \cdot (a^{-1} \cdot b^{-1}) = (a \cdot a^{-1}) \cdot (b \cdot b^{-1}) = 1 \cdot 1 = 1$$

Somit besitzt $a \cdot b$ ein Inverses, nämlich

$$(a \cdot b)^{-1} = a^{-1} \cdot b^{-1}$$

und es gilt $a \cdot b \in \mathbb{Z}_n^*$.

- (ii) Zu $a \in \mathbb{Z}_n^*$ gibt es ein inverses Element a^{-1} und es gilt:

$$a \cdot b = a \cdot c \implies a^{-1} \cdot (a \cdot b) = a^{-1} \cdot (a \cdot c)$$

$$\implies (a^{-1} \cdot a) \cdot b = (a^{-1} \cdot a) \cdot c$$

$$\implies 1 \cdot b = 1 \cdot c$$

$$\implies b = c$$

- (iii) Seien $a, b \in \mathbb{Z}_n \setminus \{0\}$ mit $a \cdot b = 0$. Wäre $a \in \mathbb{Z}_n^*$, so besäße a das Inverse a^{-1} und mit diesem folgte

$$b = 1 \cdot b = (a^{-1} \cdot a) \cdot b = a^{-1} \cdot (a \cdot b) = a^{-1} \cdot 0 = 0$$

im Widerspruch zur Annahme $b \in \mathbb{Z}_n \setminus \{0\}$. Es gilt also $a \notin \mathbb{Z}_n^*$ und ebenso folgt $b \notin \mathbb{Z}_n^*$.

(iv) Sei $a \in \mathbb{Z}_n$ mit $g := \text{ggT}(a, n) \neq 1$. Dann ist $b := \frac{n}{g} \in \mathbb{Z}_n \setminus \{0\}$ und es gilt:

$$(a \cdot b) \bmod n = \left(a \cdot \frac{n}{g}\right) \bmod n = \left(\frac{a}{g} \cdot n\right) \bmod n = 0$$

Mit (iii) folgt daher $a \notin \mathbb{Z}_n^*$.

■

In Abschnitt 2.5 wird gezeigt, dass auch die Umkehrung von Satz 2.5 (iv) gilt, d. h., ein Element $a \in \mathbb{Z}_n$ ist genau dann invertierbar, falls $\text{ggT}(a, n) = 1$ gilt. Darüber hinaus werden wir sehen, wie mithilfe des erweiterten Euklid'schen Algorithmus Inverse effizient berechnet werden können.

Frage 11

Bestimmen Sie alle invertierbaren Elemente in $\mathbb{Z}_{26}$ und ihre jeweiligen Inversen.

2.3 Matrizen über $\mathbb{Z}_n$

Zur Berechnung der Determinante einer (N, N) -Matrix

$$\mathbf{A} = (a_{ij})_{1 \leq i, j \leq N}$$

müssen Produkte über die Elemente verschiedener Teilmengen der Menge aller Koeffizienten $\{a_{ij} | 1 \leq i, j \leq N\}$ berechnet werden, diese teilweise (additiv) invertiert und schließlich aufsummiert werden. Dies ergibt sich direkt aus dem Laplace'schen Entwicklungssatz zur Determinantenberechnung. Eine Konsequenz ist, dass Determinanten von Matrizen mit ganzzahligen Einträgen ganze Zahlen sind.

Eine weitere Konsequenz ist, dass Determinanten auch für quadratische Matrizen mit Koeffizienten aus $\mathbb{Z}_n$ in $\mathbb{Z}_n$ berechnet werden können, da Produkte, Summen und Inverse bezüglich der Addition in $\mathbb{Z}_n$ berechenbar sind. Mit Satz 2.4 folgt:

Satz 2.6

Ist $\mathbf{A} = (a_{ij})_{1 \leq i, j \leq N}$ eine Matrix mit ganzzahligen Koeffizienten $a_{ij} \in \mathbb{Z}$, so gilt für $n \in \mathbb{N}$

$$\det(\mathbf{A}) \bmod n = \det\left((a_{ij} \bmod n)_{1 \leq i, j \leq N}\right)$$

wobei die Determinante auf der rechten Seite in $\mathbb{Z}_n$ berechnet wird.

Kommentar Konkret lässt sich per Induktion folgende Formel beweisen:

$$\det(\mathbf{A}) = \sum_{\sigma \in \Sigma_N} \text{sgn}(\sigma) \cdot a_{1\sigma(1)} \cdots a_{N\sigma(N)}$$

Hierbei bezeichnet Σ_N die Menge der *Permutationen* auf $\mathcal{M}_N := \{1, \dots, N\}$, d. h.

$$\Sigma_N := \{\sigma : \mathcal{M}_N \rightarrow \mathcal{M}_N \mid \sigma \text{ ist bijektiv}\}$$

und $\text{sgn}(\sigma)$ das *Signum* der Permutation σ

$$\text{sgn}(\sigma) := \det(\mathbf{M}_\sigma)$$

wobei $\mathbf{M}_\sigma$ die durch σ definierte *Permutationsmatrix* mit

$$\mathbf{M}_\sigma \cdot \mathbf{e}_i = \mathbf{e}_{\sigma(i)} \quad \text{für alle Einheitsvektoren } \mathbf{e}_i \ (i = 1, \dots, n)$$

bezeichnet. Für alle Permutationen gilt $\text{sgn}(\sigma) \in \{1, -1\}$. ◀

Aus der Gültigkeit des Determinantenmultiplikationssatzes für Matrizen mit ganzzahligen Einträgen folgt die Gültigkeit des Determinantenmultiplikationssatzes für Matrizen mit Einträgen in $\mathbb{Z}_n$:

Sind $\mathbf{A} = (a_{ij})_{1 \leq i, j \leq N}$ und $\mathbf{B} = (b_{ij})_{1 \leq i, j \leq N}$ Matrizen mit ganzzahligen Einträgen, so gilt:

$$\begin{aligned} \det(\mathbf{A} \cdot \mathbf{B}) \bmod n &= (\det(\mathbf{A}) \cdot \det(\mathbf{B})) \bmod n \\ &= \left((\det(\mathbf{A}) \bmod n) \cdot (\det(\mathbf{B}) \bmod n) \right) \bmod n \\ &= (\det(\mathbf{A}) \bmod n) \cdot_{\mathbb{Z}_n} (\det(\mathbf{B}) \bmod n) \end{aligned}$$

Mit dem Determinantenmultiplikationssatz folgt, dass die Determinante einer invertierbaren Matrix über $\mathbb{Z}_n$ ein invertierbares Element in $\mathbb{Z}_n$ sein muss.

Ist andererseits die Determinante einer quadratischen Matrix $\mathbf{A}$ über $\mathbb{Z}_n$ ein invertierbares Element in $\mathbb{Z}_n$, so ist auch $\mathbf{A}$ invertierbar und die Inverse lässt sich mit Hilfe der Adjunkten von $\mathbf{A}$ berechnen:

$$\mathbf{A}^{-1} = (\det(\mathbf{A}))^{-1} \cdot \text{adj}(\mathbf{A}) \quad (2.11)$$

Beachten Sie, dass die Adjunkte einer Matrix über $\mathbb{Z}_n$ tatsächlich über $\mathbb{Z}_n$ berechnet werden kann, da lediglich Summen und Differenzen von Produkten berechnet werden müssen, deren Faktoren Elemente aus $\mathbf{A}$ sind.

Frage 12

Zeigen Sie, dass die in Frage 7 angegebene Matrix

$$\mathbf{K} = \begin{pmatrix} 10 & 12 & 21 \\ 3 & 21 & 11 \\ 23 & 20 & 21 \end{pmatrix}$$

invertierbar ist und berechnen Sie die Inverse $\mathbf{K}^{-1}$ von $\mathbf{K}$.

Frage 13

Der Geheimtext

LJNJNUJNJNYBNZXDVLLVARJQUAFMIKDQNXJBRVNYUTCSUMYTCSZTIGMB

ist mit dem Hill-Verschlüsselungsverfahren mit einer $(3,3)$ -Matrix über $\mathbb{Z}_{26}$ berechnet worden. Der Klartext beginnt mit den Zeichen LITERATUREMP. Bestimmen Sie die zur Verschlüsselung benutzte Matrix. Bestimmen Sie außerdem den gesamten Klartext. (Hierfür können Sie CrypTool verwenden.)

2.4 Der erweiterte Euklid'sche Algorithmus

Der erweiterte Euklid'sche Algorithmus hat eine große Relevanz im Zusammenhang mit asymmetrischen kryptographischen Verfahren. Insbesondere liefert er ein effizientes Verfahren zur Berechnung von inversen Elementen in $\mathbb{Z}_n^*$ und einen Beweis von Satz 2.10 im folgenden Abschnitt 2.5.

Mithilfe des Euklid'schen Algorithmus kann der größte gemeinsame Teiler zweier ganzer Zahlen effizient berechnet werden, ohne dass die Primfaktoren der Zahlen bekannt sein müssen. Zunächst werden die Definition und einige elementare Eigenschaften des größten gemeinsamen Teilers wiederholt und im folgenden Satz 2.7 zusammengefasst (vgl. mit Studienheft Ma1-02, Abschnitt 2.2.).

Größter gemeinsamer Teiler (ggT)

Sind $a, b \in \mathbb{Z}$ mit $a \neq 0$ oder $b \neq 0$, so gibt es offensichtlich eine größte natürliche Zahl, die sowohl a als auch b teilt. Diese Zahl wird *größter gemeinsamer Teiler* von a und b genannt. In Zeichen:

$$\text{ggT}(a, b) := \max\{t \in \mathbb{N} \mid t \mid a \text{ und } t \mid b\}$$

Für den Sonderfall $a = b = 0$ wird definiert:

$$\text{ggT}(0, 0) := 0$$

Ist $\text{ggT}(a, b) = 1$, so werden a und b *teilerfremd* genannt.

Satz 2.7

Sind $a, b \in \mathbb{Z}$, so gilt:

- (i) $\text{ggT}(a, b) = \text{ggT}(b, a)$
- (ii) $\text{ggT}(a, 0) = |a|$
- (iii) $\text{ggT}(a, b) = \text{ggT}(-a, b)$
- (iv) $\text{ggT}(a, b) = \text{ggT}(a + v \cdot b, b) = \text{ggT}(a, b + v \cdot a)$ für alle $v \in \mathbb{Z}$
- (v) $\text{ggT}(a, b) = \text{ggT}(a \bmod b, b)$ für $b \neq 0$

Beweis (i), (ii) und (iii) folgen direkt aus der Definition des größten gemeinsamen Teilers.

Zum Beweis von (iv) genügt es offensichtlich aus Symmetriegründen (d.h. wegen (i)) die erste Gleichung zu verifizieren. Hierzu zeigen wir, dass für $g = \text{ggT}(a, b)$ und $h = \text{ggT}(a + v \cdot b, b)$ sowohl $g \leq h$ als auch $h \leq g$ gilt.

Aus $g \mid a$ und $g \mid b$ folgt $g \mid (a + v \cdot b)$. Somit ist g ein Teiler von $(a + v \cdot b)$ und von b , also gilt in der Tat $g \leq h$. Andererseits ist h ein Teiler von $(a + v \cdot b) - v \cdot b = a$, sodass h sowohl a als auch b teilt und daher $h \leq g$ gilt.

Schließlich folgt (v) sofort aus (iv), da $a \bmod b = a + v \cdot b$ mit $v = -a /_{\mathbb{Z}} b$ ist. ■

Wiederholen wir nun als nächstes den bereits im Studienheft Ma1-02 behandelten Euklid'schen Algorithmus, wobei der entscheidende Reduktionsschritt auf der Gleichung in Satz 2.7 (v) beruht.

Euklid'scher Algorithmus

Für gegebene $a_0, b_0 \in \mathbb{Z}$ liefert der Euklid'sche Algorithmus den größten gemeinsamen Teiler $g = \text{ggT}(a_0, b_0)$.

Bei der folgenden Beschreibung des Algorithmus sind a , b , r und g als Variablen im Sinne einer Programmiersprache aufzufassen, die Zahlen aus $\mathbb{Z}$ speichern können. Die Zuweisung eines Werts (oder eines durch eine Variable gespeicherten Werts) zu einer Variablen wird durch das Zeichen $:=$ beschrieben.

- (1) Setze: $a := a_0$ und $b := b_0$
- (2) Ist $b = 0$, so setze $g := |a|$ und beende den Algorithmus.
- (3) Setze: $r := a \bmod b$
- (4) Setze: $a := b$
- (5) Setze: $b := r$
- (6) Gehe zu (2).

Kommentar Der bereits seit weit mehr als 2000 Jahre bekannte Euklid'sche Algorithmus wird von D. Knuth¹ im zweiten Band seines mehrbändigen Werks *The Art of Computer Programming* insbesondere auch bezüglich seines Laufzeitverhaltens intensiv behandelt. Auch findet sich dort eine ausführliche Beschreibung und Diskussion des *binären ggT-Algorithmus*, welcher eine Alternative zum Euklid'schen Algorithmus darstellt. Der binäre ggT-Algorithmus ist insbesondere bei Nutzung des Stellenwertsystems zur Basis 2 einfach zu implementieren und effizient auszuführen. Bemerkenswerter Weise wurde der binäre ggT-Algorithmus erstmals von J. Stein² erst im Jahr 1967 veröffentlicht. ◀

Beispiel 2.4

Mit dem Euklid'schen Algorithmus soll $\text{ggT}(1224, 156)$ berechnet werden. Dies lässt sich schriftlich wie folgt durchführen.

- (1): Zuerst werden

$$a = a_0 = 1224 \quad \text{und} \quad b = b_0 = 156$$

untereinander geschrieben:

1224

156

- (2): $b \neq 0$, also weiter mit (3).

- (3): Zur Berechnung von $a \bmod b$ wird zunächst

$$q = a /_{\mathbb{Z}} b = 1224 /_{\mathbb{Z}} 156 = 7$$

neben b in eine weitere Spalte geschrieben:

¹ Donald Ervin Knuth (* 1938 Milwaukee) ist nicht nur der Verfasser umfangreicher theoretischer Arbeiten zur Analyse von Algorithmen, sondern auch der Erfinder des TeX-Textsatzsystems (Grundlage von LaTeX). Von seinem epochalen Werk *The Art of Computer Programming* sind bisher vier (von aktuell acht geplanten) Bänden erschienen.

² Josef Stein

a, b	q
1224	
156	7

Nun lässt sich bequem

$$r = a \bmod b = a - q \cdot b = 1224 - 156 \cdot 7 = 132$$

berechnen. Das Ergebnis schreiben wir unterhalb des Wertes von b :

a, b, r	q
1224	
156	7
132	

(4), (5): Nach den Zuweisungen in **(4)** und **(5)** besitzen a und b die Werte, die in den letzten beiden Einträge der ersten Spalte stehen, d. h. $a = 156$ und $b = 132$.

$\dots, a, b$	q
1224	
156	7
132	

(6), (2): $b \neq 0$, also weiter mit **(3)**.

(3): Nacheinander werden

$$q = a /_{\mathbb{Z}} b = 156 /_{\mathbb{Z}} 132 = 1$$

und

$$r = a - q \cdot b = 156 - 132 \cdot 1 = 24$$

berechnet und zur Erweiterung der begonnen Tabelle benutzt:

$\dots, a, b, r$	q
1224	
156	7
132	1
24	

(4), (5):

$\dots, a, b$	q
1224	
156	7
132	1
24	

(6), (2): $b \neq 0$, also weiter mit **(3)**.

(3), (4), (5): Nacheinander werden

$$q = a /_{\mathbb{Z}} b = 132 /_{\mathbb{Z}} 24 = 5$$

und

$$r = a - q \cdot b = 132 - 24 \cdot 5 = 12$$

berechnet und zur Erweiterung der begonnen Tabelle benutzt:

$\dots, a, b$	q
1224	
156	7
132	1
24	5
12	

(6), (2): $b \neq 0$, also weiter mit (2).

(3), (4), (5): Nacheinander werden

$$q = a /_{\mathbb{Z}} b = 24 /_{\mathbb{Z}} 12 = 2$$

und

$$r = a - q \cdot b = 24 - 12 \cdot 2 = 0$$

berechnet und zur Erweiterung der begonnenen Tabelle benutzt:

$\dots, a, b$	q
1224	
156	7
132	1
24	5
12	2
0	

(6), (2): Nun ist $b = 0$ (letzter Eintrag in der ersten Spalte der Tabelle) und der vorletzte Eintrag in der ersten Spalte der Tabelle liefert:

$$\text{ggT}(1224, 156) = g = a = 12$$

Beispiel 2.5

Der größte gemeinsame Teiler von -3885 und 700 soll berechnet werden.

Der Euklid'sche Algorithmus liefert mit der in Beispiel 2.4 eingeführten Schreibweise

$\dots, a, b$	q
-3885	
700	-6
315	2
70	4
35	2
0	

und mit dem vorletzten Eintrag in der ersten Spalte erhalten wir $\text{ggT}(-3885, 700) = 35$.

Sind $a, b \in \mathbb{N}$ mit $a \geq b$, so gilt

$$a \bmod b < \frac{a}{2} \quad (2.12)$$

und deshalb auch:

$$l_2(a \bmod b) < l_2(a) \quad (2.13)$$

Begründungen für (2.12) und (2.13) sollten Sie selbstständig finden!

Die Ungleichung (2.13) kann als Schlüssel zur Begründung der in der folgenden Frage formulierten Aussage zum Laufzeitverhalten des Euklid'schen Algorithmus in einem Induktionsbeweis genutzt werden.

Frage 14

Es seien $a_0, b_0 \in \mathbb{N}_0$ mit $a_0 \geq b_0$. Zeigen Sie, dass bei der Durchführung des Euklid'schen Algorithmus zur Berechnung von $\text{ggT}(a_0, b_0)$ die Folge der Schritte (3), (4), (5) und (6) höchstens

$$l_2(a_0) + l_2(b_0) - 1$$

mal durchlaufen werden.

Eine erweiterte Form des Euklid'schen Algorithmus liefert die Darstellung des größten gemeinsamen Teilers als Linearkombination der Ausgangszahlen mit ganzzahligen Koeffizienten. Hierzu wird der Euklid'sche Algorithmus anstelle von a und b mit den trivialen Linearkombinationen

$$1 \cdot a + 0 \cdot b = a \quad (2.14)$$

und

$$0 \cdot a + 1 \cdot b = b \quad (2.15)$$

gestartet. Bevor die formale Beschreibung des erweiterten Euklid'schen Algorithmus festgehalten wird, schauen wir uns die Vorgehensweise anhand eines Beispiels an.

Beispiel 2.6

Für die bereits in Beispiel 2.4 betrachteten Zahlen $a_0 = 1224$ und $b_0 = 156$ soll eine Linearkombination der Form

$$\alpha \cdot 1224 + \beta \cdot 156$$

für den bereits berechneten größten gemeinsamen Teiler $\text{ggT}(1224, 156) = 12$ bestimmt werden.

Hierzu erweitern wir die in Beispiel 2.4 eingeführte tabellarische Darstellungsform des Euklid'schen Algorithmus um zwei Spalten, sodass für die ersten drei Zahlen α, β, γ jeder Zeile gilt:

$$\alpha \cdot 1224 + \beta \cdot 156 = \gamma \quad (*)$$

Als Überschriften für die ersten beiden neuen Spalten werden die Faktoren $a_0 = 1224$ und $b_0 = 156$ aus (*) gewählt.

Für die ersten beiden Zeilen ist $\gamma = a$ bzw. $\gamma = b$ und die erweiterte Tabelle wird mit den trivialen Linearkombinationen (2.14) und (2.15) initialisiert:

1224	156	a, b	q
1	0	1224	
0	1	156	

Wie in Beispiel 2.4 wird $q = a /_{\mathbb{Z}} b = 1224 /_{\mathbb{Z}} 156 = 7$ berechnet:

1224	156	a, b	q
1	0	1224	
0	1	156	7

In der erweiterten Form des Euklid'schen Algorithmus wird im folgenden Schritt nicht nur von a der q -fache Wert von b abgezogen, sondern es werden auch jeweils das q -fache der ersten beiden Einträge der aktuellen Zeile von den entsprechenden Einträgen in der darüber liegenden Zeile abgezogen. Werden die ersten drei Einträge der nächsten Zeile als Zeilenvektor zusammengefasst, so berechnen sie sich also zu:

$$(1, 0, 1224) - 7 \cdot (0, 1, 156) = (1, -7, 132)$$

Die Tabelle mit der dritten Zeile, in der auch bereits der nächste Wert für $q = 156 /_{\mathbb{Z}} 132 = 1$ eingetragen ist, hat dann folgende Gestalt:

1224	156	$\dots, a, b$	q
1	0	1224	
0	1	156	7
1	-7	132	1

Die ersten drei Einträge der nächsten Zeile berechnen sich zu

$$(0, 1, 156) - 1 \cdot (1, -7, 132) = (-1, 8, 24)$$

und der Wert für q in der neuen Zeile wird wiederum wie zuvor in Beispiel 2.4 berechnet, d. h. $q = 132 /_{\mathbb{Z}} 24 = 5$:

1224	156	$\dots, a, b$	q
1	0	1224	
0	1	156	7
1	-7	132	1
-1	8	24	5

Die wiederholte Anwendung des Verfahrens liefert für die ersten drei Einträge der nächste Zeile

$$(1, -7, 132) - 5 \cdot (-1, 8, 24) = (6, -47, 12)$$

und $q = 24 /_{\mathbb{Z}} 12 = 2$:

1224	156	$\dots, a, b$	q
1	0	1224	
0	1	156	7
1	-7	132	1
-1	8	24	5
6	-47	12	2

Die wiederholte Anwendung des Verfahrens liefert zunächst:

$$(-1, 8, 24) - 2 \cdot (6, -47, 12) = (-13, 102, 0)$$

Da jetzt der dritte Eintrag des Vektors gleich null ist, wird die Berechnung jetzt abgebrochen. Dies vermerken wir noch durch Eintrag der berechneten Zahl 0 in der dritten Spalte:

1224	156	$\dots, a, b$	q
1	0	1224	
0	1	156	7
1	-7	132	1
-1	8	24	5
6	-47	12	2
		0	

Bitte beachten Sie, dass (*) tatsächlich für die drei ersten Elemente aller (vollständigen) Zeilen der Tabelle gilt:

Machen Sie sich hierzu zunächst klar, dass (*) ist äquivalent dazu ist, dass $\begin{pmatrix} \alpha \\ \beta \\ \gamma \end{pmatrix}$ auf dem Vektor

$$\mathbf{v}_{a_0, b_0} := \begin{pmatrix} 1224 \\ 156 \\ -1 \end{pmatrix}$$

senkrecht steht. D. h., (*) ist äquivalent zu

$$\begin{pmatrix} \alpha \\ \beta \\ \gamma \end{pmatrix} \perp \mathbf{v}_{a_0, b_0} \quad (**)$$

und (**) gilt für die ersten beiden Zeilen der Tabelle per Konstruktion und für die folgenden (per Induktion), da eine Linearkombination von Vektoren, die senkrecht auf dem Vektor $\mathbf{v}_{a_0, b_0}$ stehen, selbst auch senkrecht auf $\mathbf{v}_{a_0, b_0}$ steht.

Aus der letzten vollständigen Zeile der Tabelle lässt sich daher die gesuchte Linearkombination direkt ablesen:

$$6 \cdot 1224 + (-47) \cdot 156 = 12$$

Ausgehend von Beispiel 2.6 sollte die folgende formale Beschreibung des erweiterten Euklid'schen Algorithmus gut verständlich sein. Diese basiert auf der bereits gegebenen Beschreibung des einfachen Euklid'schen Algorithmus und verwendet zusätzliche Variablen $\alpha_a, \alpha_b, \alpha_r, \beta_a, \beta_b, \beta_r$ für die Speicherung von Einträgen in den ersten beiden Spalten der erweiterten Tabelle. Um die Darstellung nicht unnötig durch Fallunterscheidungen zu verkomplizieren, nehmen wir im Folgenden an, dass die gegebenen Zahlen a_0 und b_0 nicht negativ sind. Wegen $\text{ggT}(a_0, b_0) = \text{ggT}(|a_0|, |b_0|)$ stellt dies keine echte Einschränkung dar.

Erweiterter Euklid'scher Algorithmus (Extended Euclidean Algorithm, EEA)

Für gegebene $a_0, b_0 \in \mathbb{N}_0$ liefert der folgende Algorithmus den größten gemeinsamen Teiler $g = \text{ggT}(a_0, b_0)$ und Zahlen $\alpha, \beta \in \mathbb{Z}$ mit:

$$\alpha \cdot a_0 + \beta \cdot b_0 = g = \text{ggT}(a_0, b_0) \quad (2.16)$$

Bei der Beschreibung des Algorithmus sind a, b, r und g sowie $\alpha, \beta, \alpha_a, \alpha_b, \alpha_r, \beta_a, \beta_b, \beta_r$ und q als Variablen im Sinne einer Programmiersprache aufzufassen, die Zahlen aus $\mathbb{Z}$ speichern können. Die Zuweisung eines Werts (oder eines durch eine Variable gespeicherten Werts) zu einer Variablen wird durch das Zeichen $:=$ beschrieben.

- (1) Setze: $a := a_0, \alpha_a := 1, \beta_a := 0, b := b_0, \alpha_b := 0, \beta_b := 1$
- (2) Ist $b = 0$, so setze $g := a, \alpha := \alpha_a, \beta := \beta_a$ und beende den Algorithmus.
- (3a) Setze: $q := a /_{\mathbb{Z}} b$
- (3b) Setze: $r := a - q \cdot b, \alpha_r := \alpha_a - q \cdot \alpha_b, \beta_r := \beta_a - q \cdot \beta_b$
- (4) Setze: $a := b, \alpha_a := \alpha_b, \beta_a := \beta_b$
- (5) Setze: $b := r, \alpha_b := \alpha_r, \beta_b := \beta_r$
- (6) Gehe zu (2).

Frage 15

Bestimmen Sie für $a = 7429$ und $b = 5888$ eine Darstellung von $g = \text{ggT}(a, b)$ als Linearkombination von a und b mit ganzzahligen Koeffizienten.

Aufgabe 1

Implementieren Sie den erweiterten Euklid'schen Algorithmus in einer Methode:

```
public static void eea(BigInteger a, BigInteger b)
```

Die Methode soll eine tabellarische Darstellung der Ausführung des erweiterten Euklid'schen Algorithmus für die durch a und b repräsentierten Zahlen ausgeben. Beispielsweise sollte der Aufruf

```
eea( new BigInteger("3576222"), new BigInteger("117624") )
```

zu folgender Ausgabe führen:

3576222	117624		q

1	0	3576222	
0	1	117624	30
1	-30	47502	2
-2	61	22620	2
5	-152	2262	10
		0	

2.5 Berechnung inverser Elemente in $\mathbb{Z}_n$

Wie am Ende von Abschnitt 2.2 angekündigt soll nun der folgende Satz mithilfe des erweiterten Euklid'schen Algorithmus bewiesen werden:

Satz 2.10

$$a \in \mathbb{Z}_n^* \iff \text{ggT}(a, n) = 1 \quad (2.17)$$

Beweis Ist $\text{ggT}(a, n) \neq 1$, so folgt $a \notin \mathbb{Z}_n^*$ mit Satz 2.5 (iv).

Sei nun $\text{ggT}(a, n) = 1$.

Mit dem erweiterten Euklid'schen Algorithmus können dann $\alpha, \beta \in \mathbb{Z}$ mit

$$\alpha \cdot n + \beta \cdot a = \text{ggT}(a, n) = 1$$

bestimmt werden. Reduktion modulo n liefert:

$$(\alpha \cdot n + \beta \cdot a) \bmod n = 1 \bmod n$$

$$(\beta \cdot a) \bmod n = 1$$

In $\mathbb{Z}_n$ besitzt a daher das Inverse

$$a^{-1} = \beta \bmod n$$

■

Beispiel 2.7

In $\mathbb{Z}_{7429}$ besitzt 5888 kein Inverses, da 7429 und 5888 nicht teilerfremd sind (vgl. mit Frage 15). ◀

Beispiel 2.8

Falls $a = 20$ in $\mathbb{Z}_{147}$ ein Inverses besitzt, soll dieses berechnet werden.

Die Anwendung des erweiterten Euklid'schen Algorithmus liefert:

147	20	$\dots, a, b$	q
1	0	147	
0	1	20	7
1	-7	7	2
-2	15	6	1
3	-22	1	6
		0	

Aus der letzten vollständigen Zeile lässt sich ablesen:

$$3 \cdot 147 + (-22) \cdot 20 = 1 = \text{ggT}(147, 20)$$

Reduktion modulo $n = 147$ liefert in $\mathbb{Z}_{147}$:

$$0 + (-22 \bmod 147) \cdot 20 = 1$$

In $\mathbb{Z}_{147}$ besitzt die Zahl 20 also das inverse Element:

$$20^{-1} = -22 \bmod 147 = 125$$

Kommentar Zur Berechnung des Inversen einer invertierbaren Zahl a aus $\mathbb{Z}_n$ wird in der mithilfe des erweiterten Euklid'schen Algorithmus bestimmten Linearkombination

$$\alpha \cdot n + \beta \cdot b = 1$$

der Koeffizient α nicht benötigt. Das bedeutet, dass die entsprechende Spalte und die mit dieser Spalte zusammenhängenden Zahlen und Berechnungen nicht benötigt werden. Das Ergebnis in Beispiel 2.8 hätten wir auch mit der reduzierten Tabelle

20	$\dots, a, b$	q
0	147	
1	20	7
-7	7	2
15	6	1
-22	1	6
	0	

erhalten.

Frage 16

Bestimmen Sie mithilfe des erweiterten Euklid'schen Algorithmus das Inverse von 20 in $\mathbb{Z}_n$ für $n = 2^{16} + 1$.

Mit Satz 2.10 ergibt sich direkt die Behauptung des folgenden Satzes:

Satz 2.11

Genau dann, falls n eine Primzahl ist, besitzt jedes Element in $\mathbb{Z}_n \setminus \{0\}$ ein multiplikativ inverses Element, d. h.:

$$\mathbb{Z}_n^* = \mathbb{Z}_n \setminus \{0\} \iff n \text{ ist Primzahl}$$

Wichtig $\mathbb{Z}_n$ ist also genau dann ein Körper, falls n eine Primzahl. Ist n eine Primzahl, so kann in $\mathbb{Z}_n$ wie in der Menge der rationalen Zahlen $\mathbb{Q}$ gerechnet werden. Insbesondere kann durch alle Zahlen $\neq 0$ dividiert werden.

2.6 Mathematischer Exkurs: *Euler'sche φ -Funktion*

Die Euler'sche φ -Funktion wird zur Bezeichnung der Anzahl invertierbarer Elemente in $\mathbb{Z}_n$ eingeführt:

Euler'sche φ -Funktion

Die Euler'sche φ -Funktion ist für alle natürlichen Zahlen definiert durch:

$$\varphi(n) := \begin{cases} |\mathbb{Z}_n^*| & \text{für } n \in \mathbb{N}, n \geq 2 \\ 1 & \text{für } n = 1 \end{cases}$$

Ist $n = p$ eine Primzahl, so gilt mit Satz 2.11:

$$\varphi(p) = p - 1$$

Allgemeiner gilt für Primzahlpotenzen:

Satz 2.12

Ist $n = p^e$ eine Primzahlpotenz mit einer Primzahl p und einem $e \in \mathbb{N}$, so gilt:

$$\varphi(p^e) = p^{e-1}(p-1) \quad (2.18)$$

Beweis $\varphi(p^e)$ ist die Anzahl der Zahlen zu p^e teilerfremden Zahlen in:

$$\mathbb{Z}_{p^e} = \{0, 1, 2, \dots, p^e - 1\}$$

Eine Zahl ist aber genau dann teilerfremd zu p^e , falls sie nicht durch p geteilt werden kann. Andererseits können die Zahlen in $\mathbb{Z}_{p^e}$, die durch p teilbar sind, leicht aufgezählt werden. Es sind dies die Zahlen:

$$0, p, 2p, 3p, \dots, (p^{e-1} - 1)p$$

Es gibt also genau p^{e-1} viele durch p teilbare Zahlen in $\mathbb{Z}_{p^e}$. Damit folgt:

$$\varphi(p^e) = p^e - p^{e-1} = p^{e-1}(p-1)$$

■

Im Studienheft Datensicherheit 03 wird der chinesischen Restsatz behandelt. Ein Beweis der im folgenden Satz behaupteten multiplikativen Eigenschaft der Euler'schen φ -Funktion wird dann unter Anwendung des chinesischen Restsatzes nachgereicht.

Satz 2.13

Für $a, b \in \mathbb{N}$ mit $\text{ggT}(a, b) = 1$ gilt:

$$\varphi(a \cdot b) = \varphi(a) \cdot \varphi(b) \quad (2.19)$$

Beweis Siehe Studienheft Datensicherheit 03, Satz 1.10. ■

Ist die Primfaktorzerlegung von $n \in \mathbb{N}$ bekannt, so kann mithilfe der Sätze 2.12 und 2.13 der Wert der Euler'schen φ -Funktion $\varphi(n)$ leicht berechnet werden:

Satz 2.14

Es sei $n \in \mathbb{N}$ mit $n > 1$. Die Primfaktorzerlegung von n sei

$$n = p_1^{e_1} \cdots p_r^{e_r}$$

mit paarweise verschiedenen Primzahlen $p_1, \dots, p_r$ und positiven Exponenten $e_1, \dots, e_r \in \mathbb{N}$. Dann gilt:

$$\varphi(n) = n \cdot \prod_{i=1}^r \left(1 - \frac{1}{p_i}\right) \quad (2.20)$$

Beweis

$$\begin{aligned} \varphi(n) &= \varphi(p_1^{e_1} \cdots p_r^{e_r}) \\ &\stackrel{(2.19)}{=} \prod_{i=1}^r \varphi(p_i^{e_i}) \\ &\stackrel{(2.18)}{=} \prod_{i=1}^r p_i^{e_i-1} (p_i - 1) \\ &= \prod_{i=1}^r p_i^{e_i} \left(1 - \frac{1}{p_i}\right) \\ &= \prod_{i=1}^r p_i^{e_i} \cdot \prod_{i=1}^r \left(1 - \frac{1}{p_i}\right) \\ &= n \cdot \prod_{i=1}^r \left(1 - \frac{1}{p_i}\right) \end{aligned}$$

■

2.7 Primfaktorzerlegung natürlicher Zahlen

Für den Beweis des Satzes über die eindeutige Primfaktorzerlegung einer natürlichen Zahl wird die elementare Aussage des folgenden Hilfssatzes benötigt, welche wiederum aus der in Gleichung (2.16) gegebenen Darstellung des größten gemeinsamen Teilers zweier ganzer Zahlen folgt.

Satz 2.15

Ist $p \in \mathbb{N}$ eine Primzahl und sind $a, b \in \mathbb{Z}$, so gilt:

$$p \mid (a \cdot b) \implies p \mid a \text{ oder } p \mid b$$

Beweis Gilt $p \mid a$, so ist die Aussage richtig. Es sei also im Folgenden $p \nmid a$. Da p eine Primzahl ist, ist dann $\text{ggT}(p, a) = 1$. Mit Gleichung (2.16) gibt es daher $\alpha, \beta \in \mathbb{Z}$ mit:

$$1 = \alpha \cdot p + \beta \cdot a$$

Somit ist:

$$b = 1 \cdot b = \alpha p b + \beta a b$$

Da die beiden Summanden auf der rechten Seite dieser Gleichung von p geteilt werden (der zweite aufgrund der Voraussetzung in Satz 2.15), wird auch die Summe, also b , von p geteilt. ■

Mit Hilfe von Satz 2.15 kann der Satz über die eindeutige Primfaktorzerlegung ganzer Zahlen leicht mit einer vollständigen Induktion bewiesen werden.

Satz 2.16: Eindeutigkeit der Primfaktorzerlegung

Jede natürliche Zahl $n > 1$ besitzt eine eindeutige Primfaktorzerlegung, d. h., es gibt eindeutig bestimmte Primzahlen $p_1, \dots, p_r$ und eindeutig bestimmte natürliche Zahlen $e_1, \dots, e_r$ mit:

$$n = p_1^{e_1} \cdots p_r^{e_r}$$

Sind die vollständigen Primfaktorzerlegungen zweier Zahlen bekannt, so kann der größte gemeinsame Teiler und das kleinste gemeinsame Vielfache dieser beiden Zahlen leicht berechnet werden. Zunächst die Definition des kleinsten gemeinsamen Vielfachen zweier Zahlen:

Kleinstes gemeinsames Vielfaches (kgV)

Sind $a, b \in \mathbb{Z}$ mit $a \neq 0$ und $b \neq 0$, so gibt es natürliche Zahlen, die sowohl von a als auch von b geteilt werden (z.B. $|a| \cdot |b|$). Die kleinste dieser Zahlen wird als *kleinstes gemeinsames Vielfaches* von a und b bezeichnet. In Zeichen:

$$\text{kgV}(a, b) := \min\{v \in \mathbb{N} \mid a \mid v \text{ und } b \mid v\}$$

Satz 2.17

Besitzen $a, b \in \mathbb{N}$ die Primfaktorzerlegungen

$$a = p_1^{e_1} \cdots p_r^{e_r}, \quad b = p_1^{f_1} \cdots p_r^{f_r}$$

mit den Primzahlen $p_1 < \cdots < p_r$ und nicht-negativen Exponenten $e_1, \dots, e_r, f_1, \dots, f_r \in \mathbb{N}_0$, so gilt

$$\text{ggT}(a, b) = p_1^{\min(e_1, f_1)} \cdots p_r^{\min(e_r, f_r)}$$

und:

$$\text{kgV}(a, b) = p_1^{\max(e_1, f_1)} \cdots p_r^{\max(e_r, f_r)}$$

Mit Satz 2.17 folgt, dass für alle $a, b \in \mathbb{N}$ gilt:

$$a \cdot b = \text{ggT}(a, b) \cdot \text{kgV}(a, b) \quad (2.21)$$

Frage 17

Beweisen Sie Gleichung (2.21).

IT-Sicherheit: Gefährdungen und Maßnahmen

3

3.1	Vertraulichkeit (Confidentiality, Secrecy, Privacy)	42
3.2	Integrität (Integrity)	43
3.3	Authentizität von Nachrichten (Message Authenticity) .	45
3.4	Authentizität von Nutzern (User Authenticity)	47
3.5	Zugriffskontrolle (Access Control)	49
3.6	Nichtabstreitbarkeit, Verbindlichkeit (Non-Repudiation) .	50
3.7	Verfügbarkeit (Availability)	51
3.8	Anonymität (Anonymity)	52

Zur Einführung und Motivation haben wir im ersten Kapitel historisch überlieferte Verschlüsselungsverfahren betrachtet. Aktuell zur Nutzung empfohlene Verschlüsselungsverfahren werden in späteren Kapiteln ausführlich behandelt. Allerdings sollten Sie nicht den Eindruck gewinnen, dass durch die Verschlüsselung von Daten, alle Aufgabenstellungen zur Gewährleistung der IT-Sicherheit von IT-Systemen und Anwendungen direkt zu lösen sind.

Bevor aktuelle Verschlüsselungsverfahren (und weitere kryptographische Verfahren) im Detail behandelt werden, soll daher in diesem Kapitel zunächst ein systematischer Überblick über die *Gefährdungen* und daraus abgeleiteten *Sicherheitszielen*, möglichen grundsätzlichen *Schutzmaßnahmen* zur Erbringung der Sicherheitsziele und schließlich konkretere technische *Mechanismen* (*Algorithmentypen*) zum Schutz von IT-Systemen gegeben werden.

Um Gefährdungen und IT-Sicherheitsziele von *Entitäten* (Menschen oder IT-Komponenten), die IT-Systeme nutzen oder selbst Komponenten solcher Systeme sind, anschaulich darzustellen, werden solche Entitäten häufig durch *Alice*, *Bob* und ggf. auch noch *Charly*, *Dave*, *Eve* und *Mallory* vertreten. Von dieser Gepflogenheit wird auch in den folgenden Abschnitten Gebrauch gemacht, in denen jeweils unter der Überschrift eines IT-Schutzziels die entsprechenden Gefährdungen und grundsätzlichen Schutzmaßnahmen behandelt werden.

3.1 Vertraulichkeit (Confidentiality, Secrecy, Privacy)

Sollen Daten nur für einen definierten Nutzerkreis zugänglich sein, so besteht das Schutzziel in der Gewährleistung der *Vertraulichkeit* (*Geheimhaltung*) der Daten. Die Gefährdung besteht darin, dass Unbefugte in den Besitz der Daten kommen.

Eine vertrauliche Informationsverarbeitung und -speicherung kann bereits für eine einzelne auf sich allein gestellte Entität von großer Bedeutung sein. Beispiele:

- Alice fürchtet für den Fall eines Verlusts ihres Notebooks, dass auch ihre persönlichen Daten in falsche Hände geraten könnten.
- Alice nutzt bei jeder Registrierung bei einer durch Benutzername und Passwort geschützten Webseite jeweils ein neues und hinreichend langes Passwort. Da sie nicht in der Lage ist, sich all die von ihr generierten verschiedenen Passwörter zu merken, benötigt sie eine Möglichkeit die Passwörter mithilfe einer speziellen Anwendung so zu speichern, dass die Geheimhaltung gewährleistet ist.

Wichtig ist die Vertraulichkeit natürlich auch bei vielen Daten (Nachrichten), die von einer Entität an eine andere, also von Alice an Bob, übertragen werden. Hier besteht die Gefährdung darin, dass die Nachricht auf dem Übertragungsweg von beispielsweise Eve *belauscht* (*erschnüffelt*) wird (eng.: *to eavesdrop* (*to sniff*)). Abbildung 3.1 stellt diese Gefährdung dar.

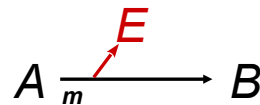


Abb. 3.1 Alice (A) sendet eine Nachricht (Message m) an Bob (B), die von Eve (E) mitgelesen wird.

Beispiele:

- Alice schickt eine persönliche Email an Bob und möchte sich sicher sein, dass diese nicht von Dritten (z. B. dem Administrator ihres Email-Ausgangsservers) mitgelesen werden kann.
- Alice benutzt einen Webbrowser um sich mit ihrem Bankserver (Bob) zu verbinden und ihre Bankgeschäfte per Homebanking abzuwickeln. Verständlicherweise wollen Alice und Bob sichergehen, dass die ausgetauschten Daten nicht von Dritten belauscht werden können. Neben den vertrauenswürdigen Kontodaten selbst trifft dies erst recht auf die Anmeldedaten von Alice zu, anhand derer der Zugang zu den Kontodaten gewährt wird.

Frage 18

Das letzte Beispiel ist bei der Festlegung der Alice zugedachten Rolle etwas ungenau. Begründen Sie dies und finden Sie eine geeignete präzisere Beschreibung von Alice.

3.1.1 Schutzmaßnahmen: Verschlüsselungsverfahren

Neben organisatorischen Maßnahmen (wie beispielsweise Vorgaben zur sicheren Verwahrung von Datenträgern oder zur Nutzung von Standalone-Rechnern und geeigneten Zugriffskontrollsystemen (vgl. mit Abschnitt 3.5)) ist *die* technisch Maßnahme zur Gewährleistung der Vertraulichkeit von Daten die Anwendung eines Verschlüsselungssystems.

Allerdings liefert die Anwendung eines Verschlüsselungsverfahrens per se noch keinen umfassenden Vertraulichkeitsschutz. Von herausragender Bedeutung ist nämlich die Berücksichtigung grundlegender Aufgabenstellungen rund um das Schlüsselmanagement. Zu nennen sind:

- *Schlüsselerzeugung*

Schlüssel sind durch einen kryptographisch sicheren Zufallszahlengenerator zu erzeugen. Jeder mögliche Schlüssel (jedes Element des zum Verschlüsselungsverfahren gehörigen Schlüsselraums $\mathcal{K}$) soll bei einer Schlüsselerzeugung mit gleicher Wahrscheinlichkeit auftreten.

- *Schlüsselspeicherung*

Schlüssel müssen sicher gespeichert werden.

- *Schlüsselaustausch*

Damit Bob eine verschlüsselte Nachricht von Alice wieder entschlüsseln kann, müssen Alice und Bob bei der Verwendung eines symmetrischen Verschlüsselungsverfahrens vorab einen Schlüssel austauschen.

Bei der Verwendung eines asymmetrischen Verschlüsselungsverfahrens muss Alice zur Verschlüsselung der Nachricht vorab den öffentlichen Schlüssel von Bob besitzen.

3.2 Integrität (Integrity)

Soll die Unversehrtheit von gespeicherten oder übertragenen Daten sichergestellt werden, so wird von dem Schutzziel der *Integrität* gesprochen. Die Gefährdung der Integrität besteht darin, dass bei der Speicherung oder Übertragung von Daten technische Fehler auftreten oder von einem Angreifer Daten mit (böser) Absicht verändert werden.

Methoden zum Schutz vor zufällig auftretenden technischen Fehlern (fehlerhafte Speicherzellen, Störsignale bei einer Datenübertragung) sind Ihnen sicherlich aus dem Gebiet der *Kommunikationstechnik* bekannt, nämlich die Verwendung von *fehlererkennenden* oder sogar *fehlerkorrigierenden Codes*.

Im Rahmen der IT-Sicherheit wird vorrangig die Gefährdung betrachtet, die von einem Angreifer ausgeht, der Daten mit (böser) Absicht verändert.

Beispiele:

- *Schadsoftware (Malware)* kann beispielsweise Daten auf einem Rechner verändern, sodass diese für den Nutzer unbrauchbar werden oder, im Fall der Modifikation von ausführbaren Dateien, weitere Schäden auf dem *infizierten* System verursachen.
- Charly gelingt es durch einen Angriff auf ein lokales Netzwerk (LAN oder WLAN) bei der Übertragung von Daten von Alice an Bob beliebige Bits auf der *Bitübertragungsschicht (Physical Layer des OSI-Modells)* nach Belieben zu modifizieren (s. Abb. 3.2).

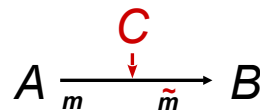


Abb. 3.2 Charly (C) ändert die Nachricht m in die Nachricht $\tilde{m}$ ab.

3.2.1 Schutzmaßnahmen: Hashfunktionen, Whitelists

Zur Gewährleistung der Integrität von Dateien auf einem IT-System können kryptographisch sichere Prüfsummen zum Einsatz kommen. Von den relevanten Dateien werden diese unter Anwendung einer kryptographischen *Hashfunktion* berechnet und in einem besonders gesicherten Bereich des IT-Systems in einer sogenannten *Whitelist* gespeichert. Vor der Verwendung einer der Dateien wird der Hashwert erneut berechnet und mit dem gespeicherten Wert verglichen. Da die Hashwerte quasi einen Fingerabdruck der Daten repräsentieren, würde eine Veränderung der Daten in einem veränderten Hashwert resultieren und bei dem Vergleich auffallen.

Als Methode zur Gewährleistung der Datenintegrität bei einer Datenübertragung kann die Nutzung von Hashwerten nur dann sinnvoll sein, wenn der Hashwert über einen anderen (sicheren) Kanal als die eigentlichen Daten übertragen werden. Wird der Hashwert wie bei einem fehlererkennenden Code nämlich den zu übertragenden Daten einfach hinzugefügt, so könnte ein Angreifer natürlich auch den Hashwert nach einer Modifikation erneut berechnen und entsprechend abändern.

Frage 19

Wieso sollte bei dem oben beschriebenen Verfahren keinesfalls ein Cyclic Redundancy Checkcode (CRC) anstelle einer Hashfunktion verwendet werden?

Zu beachten ist, dass bei der Datenübertragung einer Nachricht m über ein unsicheres Netzwerk Hashfunktionen keinen Integritätsschutz im Sinne der IT-Sicherheit bieten. Wird der Hashwert $h(m)$ der Nachrichten m zusammen mit m (über das gleiche Netzwerk) übertragen, so kann eine Angreiferin natürlich ebenfalls für eine von ihr abgeänderte oder neu eingespielte Nachricht $\tilde{m}$ den passenden Hashwert $h(\tilde{m})$ berechnen und übertragen.

Frage 20

Wieso ist die aus der Kommunikationstechnik bekannte Verwendung von fehlererkennenden Codes (z. B. Cyclic Redundancy Checkcodes (CRC)) selbst dann nicht zum Schutz vor gezielten Veränderungen geeignet, wenn ein Angreifer nicht Daten unbemerkt in den Übertragungskanal einspielen kann, sondern nur ausgewählte einzelne Bits einer übertragenen Nachricht in Echtzeit abändern kann?

3.3 Authentizität von Nachrichten (Message Authenticity)

Die im letzten Abschnitt beschriebene Gefährdung der Integrität bei der Nutzung eines unsicheren Kanals zur Datenübertragung durch Modifikation der Daten (Abb. 3.2) stellt nur eine Angriffsvariante eines aktiven Angreifers, d. h. eines Angreifers der Daten nicht nur mitleesen sondern auch verändern oder einspielen kann, dar. Weitere Bedrohungen, die bei einer Übertragung von Daten von Alice an Bob von dem aktiven Angreifer Charly ausgehen, sind:

- Charly kann beliebige Daten an Bob schicken (s. Abb. 3.3).
- Charly kann beliebige Daten an Bob schicken und außerdem die reguläre Datenübertragung von Alice an Bob belauschen. Insbesondere kann Charly bereits von Alice an Bob übertragene Daten in einem sogenannten *Replay-Angriff* wiederholt an Bob schicken (s. Abb. 3.4).
- Charly kann die reguläre Datenübertragung von Alice an Bob unterbrechen, die von Alice an Bob geschickten Daten lesen und beliebige Daten an Bob verschicken. Charly kann sich unter diesen Voraussetzungen also quasi zwischen Alice und Bob positionieren und Nachrichten von Alice an Bob lesen und nach belieben (modifiziert) weiterleiten und zusätzliche Nachrichten im Namen von Alice (Impersonation) an Bob senden. In einem solchen Szenario wird von einem *Man-In-The-Middle-Angriff (MITM)* gesprochen (s. Abb. 3.5).

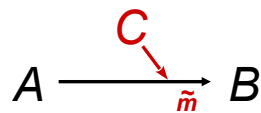


Abb. 3.3 Charly (C) sendet die Nachricht $\tilde{m}$ an Bob (B).

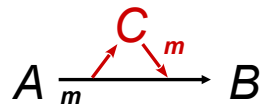


Abb. 3.4 Charly (C) sendet die Nachricht m zu einem späteren Zeitpunkt nochmals an Bob (B).

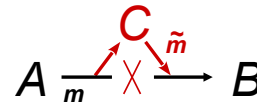


Abb. 3.5 Charly (C) unterbricht den direkten Übertragungsweg von Alice nach Bob, kann alle Nachrichten von Alice an Bob lesen und beliebige Nachrichten an Bob senden (MITM-Angriff).

Die grundsätzliche Problematik bei den skizzierten Angriffsszenarien besteht darin, dass Bob keine Möglichkeit hat, zu entscheiden, ob eine empfangene Nachricht überhaupt von Alice verschickt wurde, auch wenn Alice als Absender in der Nachricht genannt ist. (Die Absenderadresse kann je nach betrachteten Protokoll zum Beispiel eine MAC-Adresse, eine IP-Adresse oder eine Email-Adresse sein.) Bob benötigt also ein Mittel um die Authentizität des Absenders einer Nachricht überprüfen zu können!

Wichtig Besitzt Bob eine Möglichkeit die Authentizität einer Nachricht zweifelsfrei feststellen zu können, so bedeutet dies, dass Bob sicher sein kann, dass die empfangene Nachricht m genau in der gleichen Form (bitweise identisch) von Alice verschickt wurde. Insofern impliziert die Authentizität einer Nachricht auch ihre Integrität im Sinne von Abschnitt 3.2. ◀

Warnung In der Literatur wird nicht immer zwischen den Begriffen Nachrichtenintegrität und Nachrichtenauthentizität unterschieden. Häufig wird der Begriff Integrität auch in dem Sinne der hier als Authentizität definierten Eigenschaft einer Nachricht benutzt. ◀

Wichtig Zur Abwehr eines Replay-Angriffs (Abb. 3.4) genügt die Feststellung der Nachrichtenauthentizität nicht! ◀

3.3.1 Schutzmaßnahmen: Signaturen, Message Authentication Codes

Prinzipiell ist die in Abschnitt 3.2.1 skizzierte Nutzung eines zweiten Kanals über den ein Hashwert der Daten (oder die Daten selbst) zusätzlich übertragen werden eine mögliche Maßnahme zur Gewährleistung der Nachrichtenauthentizität, falls Bob (mit hoher Wahrscheinlichkeit) davon ausgehen kann, dass tatsächlich Alice die Quelle des zweiten Kanals darstellt. (Andernfalls könnte Charly beide Kanäle im Namen von Alice nutzen.) Beispielsweise wird ein solcher zweiter Kanal bei einigen Homebanking-Anwendungen verwendet, bei denen die mithilfe eines Webbrowsers übertragenen Daten nochmals über eine spezielle eigenständige Smartphone-App bestätigt werden müssen.

In der Regel steht ein sicherer zweiter Kanal nicht zur Verfügung. Um die Authentizität von Nachrichten trotzdem sicherzustellen können anstelle der in Abschnitt 3.2.1 beschriebenen Hashfunktionen, entweder digitale *Signaturen* oder sogenannte *Message Authentication Codes* (MAC-Verfahren) eingesetzt werden.

Signaturverfahren

Bei der Verwendung eines Signaturverfahrens S berechnet Alice für die Nachricht m mit ihrem privaten Schlüssel $k_{A,pri}$ eine Signatur

$$s_A(m) := S_{k_{A,pri}}(m) \quad (3.1)$$

und sendet diese zusammen mit der Nachricht m an Bob. Unter Verwendung des zu $k_{A,pri}$ gehörenden öffentlichen Schlüssels $k_{A,pub}$ und dem zu S korrespondierenden Verifikationsverfahrens V kann Bob anhand des Rückgabewerts

$$V_{k_{A,pub}}(m, s_A(m)) \quad (3.2)$$

überprüfen, ob die Signatur tatsächlich korrekt ist, und damit die Authentizität der Nachricht bestätigen. (Der Wert in (3.2) ist genau dann TRUE, falls (3.1) gilt.)

MAC-Verfahren

Bei der Verwendung eines MAC-Verfahrens M müssen sich Alice und Bob zunächst einmalig auf einen geheimen Schlüssel $k_{A,B}$ einigen. Alice kann dann für jede Nachricht m den MAC-Wert

$$h_{A,B}(m) := M_{k_{A,B}}(m) \quad (3.3)$$

berechnen und zusammen mit der Nachricht m an Bob senden. Nach Empfang von Nachricht m und MAC-Wert h , berechnet Bob den MAC-Wert entsprechend (3.3) selbstständig und vergleicht das Ergebnis mit dem empfangenen Wert. Sind die Werte gleich, so kann Bob davon ausgehen, dass die Nachricht authentisch ist, da nur Alice als einzige weitere Besitzerin von $k_{A,B}$, den richtigen MAC-Wert für m berechnen konnte.

Schutz vor Replay-Angriffen

Zum Schutz vor Replay-Angriffen wird kein neues kryptographisches Verfahren benötigt. Vielmehr ist das Protokoll zur Nachrichtenübertragung geeignet zu erweitern, sodass Bob erkennen kann, ob eine empfangene Nachricht ein Duplikat ist. Hierzu kann beispielsweise jede Nachricht um einen *Zählerwert* (*Counter*) erweitert werden, bevor eine Signatur oder ein MAC-Wert berechnet werden. Wird der Zähler mit jeder neuen Nachricht inkrementiert, so muss Bob nach der Verifizierung der Authentizität nur noch überprüfen, ob der in der Nachricht enthaltene Zählerwert größer als der Wert der zuvor empfangenen Nachricht ist.

Falls Alice und Bob den Wert des Nachrichtenzählers ohne dessen Übertragung synchron halten können, erübrigt sich übrigens dessen Übertragung, da es genügt, ihn nur zur Berechnung der Signatur bzw. des Hashwerts der Nachricht hinzuzufügen.

Alternativ kann eine Nachricht um einen Zeitstempel ergänzt werden. Bob kann dann mithilfe eines Zeitvergleichs feststellen, ob die Nachricht älteren Datums ist und als Replay anzusehen ist.

3.4 Authentizität von Nutzern (User Authenticity)

Neben dem im letzten Abschnitt behandelten Aspekt der Authentizität von Nachrichten, gibt es den Aspekt der Authentifikation von Nutzern eines IT-Systems oder einer IT-Anwendung. Ein Nutzer kann hierbei eine Person oder aber auch ein anderes IT-System bzw. eine andere IT-Anwendung sein.

Hierzu einige Beispiele:

- Alice kann sich an ihrem Arbeitsplatzrechner mit einem Passwort, mit ihrem Fingerabdruck an einem Fingerabdruckleser oder mit einer Smartcard anmelden.
- Alice kann ihr Smartphone durch das Zeichnen eines von ihr zuvor festgelegten Musters entsperren. Außerdem wird das Smartphone automatisch entsperrt, wenn es sich in der Nähe ihres Laptops befindet und dessen Bluetooth-Funktionalität eingeschaltet ist.
- Alice verwendet ihre Bankkarte um an einem Bankautomaten ihren Kontostand zu überprüfen und Geld abzuheben.
- Beim Aufruf einer Webseite über das HTTPS-Protokoll authentifiziert sich der Webserver gegenüber dem Webbrowser.

Kommentar Im Prinzip kann die Nutzerauthentifikation als spezielle Art einer Nachrichtenauthentifikation betrachtet werden, bei der die Nachricht nicht relevant ist. (Authentifikation der leeren Nachricht.) Der Kanal über den sich Alice gegenüber Bob authentifiziert ist hierbei häufig eine technische Schnittstelle (Fingerabdruckleser, etc.). Besondere Beachtung verdienen in diesem Zusammenhang Gefährdungen, die möglicherweise von Replay-Angriffen ausgehen können. ◀

3.4.1 Schutzmaßnahmen

Möchte sich Alice gegenüber Bob authentifizieren, so benötigt Alice hierfür ein *Authentifizierungsmittel*, welches quasi als Schlüssel zur Freischaltung eines Interaktionskanals dient. Häufig wird zwischen folgenden Authentifizierungsmitteln unterschieden:

- *Wissen* eines Geheimnisses, welches sowohl Alice als auch Bob kennen. Bei menschlichen Nutzern sind dies typischerweise Passwörter, PINs oder Entsperrmuster.

Bei technischen Komponenten (beispielsweise WLAN-Client und Access Point) kommen symmetrische Schlüssel $k_{A,B}$ (Passwörter über der Zeichenmenge aller möglichen Bytewerte) zum Einsatz. Solche Schlüssel müssen natürlich vor einer Nutzung konfiguriert werden und werden daher auch als *Preshared Keys* bezeichnet.

Findet die Authentifikation über einen unsicheren Kanal (z. B. bei einer WLAN-Anmeldung) statt, so kann der Schlüssel natürlich nicht direkt übertragen werden. Hier kommen daher *Challenge-Response-Protokolle* zum Einsatz, bei denen Alice zunächst einen Authentifizierungswunsch an Bob schickt und Bob mit einer *Challenge* (häufig eine Zufallszahl) antwortet. Alice wiederum berechnet aus der Challenge unter Verwendung des Schlüssels $k_{A,B}$ eine *Response* (z. B. einen MAC-Wert der Challenge) und schickt diese an Bob. Da Bob auch im Besitz von $k_{A,B}$ ist, kann er die Response ebenfalls berechnen und mit der von Alice empfangenen vergleichen.

- *Besitz* eines spezifischen Schlüssels, den nur Alice besitzt. Beispiele sind:
 - Ein solcher Schlüssel ist idealerweise ein spezifisches hardwarebasiertes *Security-Token*. Beispielsweise können Smartcards oder NFC-Tags als ein solches Security-Token dienen.
 - Um eine Authentifikation von IT-Komponenten zu ermöglichen, können diese bei der Produktion bereits mit einem *Trusted Plattform Modul (TPM)* ausgestattet werden.
 - Häufig kommen auch andere Geräte als Schlüssel zum Einsatz. Insbesondere sind hier Smartphones mit einer spezifisch personalisierten App zu nennen.
 - Auf Grundlage der im Bluetooth-Standard definierten *Pairing*-Mechanismen können beliebigen Geräten mit einer Bluetooth-Schnittstelle als Authentifizierungsmittel gegenüber anderen Geräten, die ebenfalls über eine Bluetooth-Schnittstelle verfügen, genutzt werden.

Zu beachten ist, dass bei all den genannten Beispielen letzten Endes wiederum die Kenntnis (*Wissen*) eines digitalen Schlüssels vorliegen muss, dieser digitale Schlüssel jedoch an eine spezifische Hardware gebunden ist. Damit von einem solchen hardwarebasierten Schlüssel keine unerlaubten Kopien angefertigt werden können, sind Vorkehrungen zu treffen, dass der digitale Schlüssel nicht (oder nur von zuvor gegenüber der Schlüssel-Hardware authentisierten Nutzern) aus der Hardware ausgelesen werden kann. Im Idealfall wird ein privater Schlüssel auf der Hardware (Smartcard, TPM) direkt erzeugt, verlässt diese niemals und wird zum Signieren von Challenges genutzt.

- Alice kann ein Schlüsselpaar mit einem geeigneten Programm auch auf ihrem Rechner oder Smartphone selbst erzeugen. Sie besitzt dann den privaten Schlüssel, der als Datei

auf ihrem Gerät abgelegt ist. Es obliegt dann ihrer Verantwortung, dass keine Kopien dieser Datei (Nachschlüssel) in die falschen Hände geraten. Zum Schutz eines solchen als Datei abgelegten Schlüssels (*Software-Token*) wird die Datei üblicherweise mit einem symmetrischen Verschlüsselungsverfahren verschlüsselt, wobei der Schlüssel aus einem von Alice zu wählenden (und zu merkenden!) Passwort abgeleitet wird. Vor einer Nutzung des Software-Tokens besteht daher immerhin die Notwendigkeit der Authentifikation mit dem Passwort.

- *Eigenschaft* von Alice, die sie charakterisiert. Bei einem menschlichen Nutzer sind dies *biometrische Merkmale* wie Fingerabdruck, Iris oder Gesichtsgeometrie.

Hardware-Komponenten gleichen Typs besitzen bezüglich ihrer physikalischen Eigenschaften eine Varianz, die im Prinzip zur Authentifikation einer spezifischen Komponente genutzt werden kann. In diesem Zusammenhang werden seit einigen Jahren sogenannte *Physical Unclonable Functions (PUF)* diskutiert, die in einer physikalischen Struktur enthalten und für Eingabewerte leicht berechenbare aber nicht vorhersagbare Funktionswerte liefern.

Vielen Anwendungen nutzen eine sogenannte *Multifaktor-Authentifikation* (meist *2-Faktor-Authentifikation*), bei der mindestens zwei verschiedene der oben genannten Authentifizierungsmittel zusammen eingesetzt werden. Beispiele hierzu:

- Möchte Alice einen Bankautomaten nutzen, so muss sie im *Besitz* einer Bankkarte sein und sie muss *wissen*, wie die korrekte PIN lautet.
- Möchte Alice ihre Bankgeschäfte per Homebanking abwickeln, muss sie zunächst bei der Anmeldung bei dem Bankportal ein Passwort eingeben. Anschließend muss sie mit ihrer Smartphone-App einen QR-Code scannen, der eine vom Bankserver aktuell generierte Zufallszahl (Challenge) enthält. Mit dem spezifischen Schlüssel, der bei der Personalisierung der App installiert wurde, wird von der App eine Signatur oder ein MAC-Wert (Response) berechnet und an den Bankserver zurückgeschickt. Nach der Verifikation der Response kann seitens der Bank davon ausgegangen werden, dass auf der anderen Seite des Kanals eine Nutzerin sitzt, die über das *Wissen* verfügt, wie das Passwort von Alice lautet, und außerdem das Smartphone von Alice in ihrem *Besitz* hat.

3.5 Zugriffskontrolle (Access Control)

Auf einem IT-System können unterschiedliche Ressourcen zur Verfügung stehen, sowohl in der Form von Daten als auch von Diensten zur Erledigung IT-spezifischer oder anwendungsbezogener Aufgaben. Auch kann ein IT-System die Schnittstelle zur Überwachung oder Steuerung physikalischer Prozesse bereitstellen, beispielsweise als Produktleitsystem zur Steuerung der Fertigung in einer Fabrik oder als Prozessleitsystem zur Steuerung einer Raffinerie oder eines Kraftwerks.

3.5.1 Schutzmaßnahme: *Zugriffskontrollsystem (Access Control System)*

Als Schutzmaßnahme gegen eine versehentliche oder absichtsvolle Fehlnutzung (Sabotage) von Ressourcen eines IT-Systems sollte jedem Nutzer nur Zugriff auf die Ressourcen gewährt werden, die dieser zur Erledigung seiner Aufgabe benötigt. Auch sind beispielsweise aus Gründen des Datenschutzes Zugriffsmöglichkeiten auf Daten nutzerspezifisch zu definieren und zu implementieren.

Zur Implementierung der oben skizzierten nutzerspezifischen Zugriffsregeln muss ein IT-System zunächst über eine Möglichkeit zur Authentifikation von Nutzern verfügen. Zusätzlich muss das IT-System über ein integriertes Zugriffskontrollsystem (Access Control System) verfügen, mit dem nutzerspezifische Zugriffsrechte festgelegt werden können. Die Konfiguration eines Zugriffskontrollsystems wird mit sogenannten *Access Control Lists (ACL)* beschrieben.

Häufig werden von IT-Systemen *rollenbasierte Zugriffskontrollsysteme (Role Based Access Control (RBAC))* implementiert. Auf einem solchen System können *Rollen* definiert werden, die den auf dem System auszuführenden Aufgaben entsprechen. Den Rollen werden dann die für die Ausführung der entsprechenden Aufgabe benötigten Zugriffsrechte zugeordnet. Schließlich werden den Nutzern des Systems die Rollen zugeordnet, die ihren auszuführenden Aufgaben entsprechen.

3.6 Nichtabstreitbarkeit, Verbindlichkeit (Non-Repudiation)

Bei der *Nichtabstreitbarkeit* handelt es sich um einen weiteren Aspekt der Nachrichtenintegrität. Hierbei geht es darum, dass der Empfänger Bob einer Nachricht von Alice, die Authentizität der Nachricht auch gegenüber Dritten nachweisen kann. Denken Sie beispielsweise an einen Vertrag der durch den Austausch entsprechender Nachrichten zwischen Alice und Bob ausgehandelt wurde. Bestreitet Alice zu einem späteren Zeitpunkt, dass sie mit Bob einen solchen Vertrag abgeschlossen hat, müsste Bob zur Durchsetzung seiner Ansprüche gegenüber einem Dritten (Gericht, etc.) nachweisen, dass Alice tatsächlich dem Vertrag zugestimmt hat. Dazu muss die Authentizität der ursprünglichen Nachrichten von Alice auch von Dritten (im Nachhinein) noch überprüfbar sein, wie dies in Abbildung (3.6) dargestellt ist. Ist dies der Fall besitzen die Nachrichten die Eigenschaft der Nichtabstreitbarkeit (Verbindlichkeit).

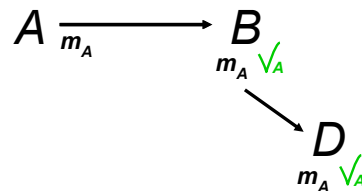


Abb. 3.6 Nichtabstreitbarkeit der Nachricht m_A durch Alice (A): Nicht nur Bob (B) sondern auch beliebige Dritte (wie z. B. Dave (D)) können die Authentizität von m_A überprüfen.

3.6.1 Schutzmaßnahme: Signaturen und PKI

Die bereits in Abschnitt 3.3.1 genannten Signaturverfahren, gewährleisten bei ihrer Nutzung nicht nur die Nachrichtenauthentizität sondern gleichzeitig auch die Nichtabstreitbarkeit. Voraussetzung hierfür ist, dass der öffentliche Schlüssel, der zur Verifikation der von Alice erstellten Signaturen benötigt wird, nicht nur Bob, sondern auch einem jeden Dritten, der die Signaturen verifizieren soll, so zur Verfügung steht bzw. zur Verfügung gestellt werden kann, dass die Zuordnung des öffentlichen Schlüssels zur Identität von Alice zweifelsfrei festgestellt werden kann.

In diesem Zusammenhang kommen digitale *Zertifikate (Certificates)* zum Einsatz, die den Zusammenhang eines öffentlichen Schlüssels mit den Angaben des Besitzers des zugehörigen

privaten Schlüssels herstellen. Öffentlicher Schlüssel und Besitzerdaten werden zusammen mit weiteren Daten von einer allseits (also von Alice, Bob und Dave) als vertrauenswürdig erachteten *Zertifizierungsstelle* (*Certification Authority, CA*) zusammengefasst, kodiert und signiert. Das Zertifikat besteht schließlich aus den signierten Daten und der Signatur. Der öffentlich Schlüssel zur Verifikation von CA-Signaturen muss natürlich auch auf vertrauenswürdigen Wege zur Verfügung stehen. (Beispielsweise als Teil der Installation eines Betriebssystems.)

Die durch den Betrieb von Zertifizierungsstellen bereitgestellte Infrastruktur wird als *Public Key Infrastructure (PKI)* bezeichnet. Zertifikate und andere im Rahmen einer PKI auszutauschende Daten werden in der Regel entsprechend dem von der *International Telecommunication Union (ITU)* verabschiedeten Standard X.509 spezifiziert und kodiert. Das Profil für X.509-konformen Zertifikaten und Sperrlisten (*Certification Revocation List (CRL)*) für Anwendungen im Internet ist als RFC 5280 (Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile) verfügbar.

Wesentliche Aufgaben einer CA im Rahmen einer PKI sind:

- Annahme und Überprüfung von Zertifizierungsanträgen (*Certification Signing Request, CSR*) bei einer Abteilung der CA, die auch als *Registration Authority (RA)* bezeichnet wird.
Ein CSR enthält bereits die den Antragsteller identifizierenden Daten und seinen öffentlichen Schlüssel. Bei der Überprüfung eines CSR muss insbesondere sichergestellt werden, dass der Antrag tatsächlich von dem im Antrag genannten Antragsteller eingereicht wurde.
- Nach erfolgreicher Überprüfung eines CSR, werden die oben genannten Datenfelder aus dem CSR zusammen mit weiteren Daten (insbesondere zur Identifikation des Zertifikats durch Angaben zur CA und einer Seriennummer sowie zur Festlegung eines Gültigkeitszeitraums) in einer definierten Weise kodiert. Von dem so erzeugten Datenblock (*to be signed certificate (tbsCertificate)*) berechnet die CA eine Signatur und fügt *tbsCertificate* und Signatur zu einem Zertifikat zusammen, welches sie schließlich an den Antragsteller schickt.
- Bereitstellung des öffentlichen CA-Schlüssels (üblicherweise in Form eines selbstsignierten *Root-Zertifikats*).
- Bereitstellung und Aktualisierung einer Sperrliste (CRL), die die Seriennummern der Zertifikate enthält, die zwar entsprechend der im Zertifikat angegebenen Gültigkeitsdauer noch benutzt werden könnten, aber wegen einer Kompromittierung des zugehörigen privaten Schlüssels von der CA vorzeitig als ungültig erklärt und daher zurückgezogen wurden.
Informationen über gesperrte Zertifikate können von einer CA auch über das *Online Certificate Status Protocol (OCSP)* zur Verfügung gestellt werden. Die Spezifikation des Protokolls findet sich in RFC 6960 (X.509 Internet Public Key Infrastructure Online Certificate Status Protocol - OCSP).

3.7 Verfügbarkeit (Availability)

Die Verfügbarkeit von Daten und IT-Systemen ist dann gegeben, falls diese wie vorgesehen genutzt werden können. Typische Bedrohungsszenarien, in denen die Verfügbarkeit verletzt ist, sind beispielsweise:

- Datenverlust durch Defekte der die Daten speichernden Systeme (z. B. durch einen Festplattencrash).
- Datenverlust durch Schadsoftware (Malware), die Daten irreversible zerstört/verschlüsselt (z. B. durch sogenannte *Ransomware*).

- Nichterreichbarkeit von Diensten in einem lokalen Netzwerk (LAN, WLAN) aufgrund eines Ausfalls von Infrastrukturkomponenten des Netzwerks. Insbesondere können solche Ausfälle durch aktive Angriffe verursacht werden, wobei zwischen Angriffen zu differenzieren ist, die aus dem lokalen Netzwerk heraus ausgeführt werden und solchen, die durch externe Angreifer über das Internet ausgeführt werden.
- Nichterreichbarkeit von Webseiten/Webdiensten aufgrund erfolgreicher *(D)DoS-Angriffe* (*(Distributed) Denial of Service*).

3.7.1 Schutzmaßnahme

Einen wirkungsvollen Schutz vor Datenverlust lassen sich durch *redundante örtlich verteilte Datenspeicherungen* erzielen. Im kleineren Maßstab kann die regelmäßige Erstellung von Datenbackups auf externen Datenträgern einen akzeptablen Schutz vor Datenverlust darstellen.

Zum Schutz vor Malware sollten auf einem PC *Virens Scanner* und *Packetfilter* (*Personal Firewall*) eingesetzt werden. Ein vollständiger Schutz wird sich durch rein technische Lösungen für einen *DAU* (*Dümmsten Anzunehmenden User*) allerdings wohl niemals realisieren lassen.

Zum Schutz von Netzwerken und Diensten, die über diese erreichbar sind, sind diese zunächst unter Berücksichtigung möglicher Angriffsszenarien so sicher wie möglich auszulegen und zu konfigurieren. Darüber hinaus ist ein mit dem Internet verbundenes Netzwerk durch den Einsatz einer robusten und sorgfältig konfigurierten externen Firewall abzusichern. Darüber hinaus kann der Einsatz von *Intrusion Detection Systemen* (*IDS*) und *Intrusion Prevention Systemen* (*IPS*) einen weitergehenden Schutz bieten. Ein vollständiger Schutz gegen DDoS-Angriffe ist auf Grundlage der eingesetzten Protokolle im Internet allerdings prinzipiell nicht realisierbar.

3.8 Anonymität (Anonymity)

Ein Interesse an Anonymität können Sender oder Empfänger einer Nachricht haben. Zum Beispiel möchte Alice als Absenderin brisanter Fakten an eine Enthüllungsplattform anonym bleiben. Oder Bob möchte als Empfänger einer illegal erpressten Bitcoin-Zahlung anonym bleiben.

Natürlich gibt es im Zusammenhang mit dem elektronischen Transfer von Zahlungen (elektronisches Geld) auch legitime Bedürfnisse nach Anonymität. Als ein weiteres Anwendungsbeispiel, bei der die Anonymität von großer Bedeutung ist, seien elektronische Wahlen genannt. Im Rahmen der Studienhefte wird das Schutzziel Anonymität und Maßnahmen zu dessen Gewährleistung allerdings nicht vertieft behandelt werden.

Verschlüsselungsverfahren

4

4.1	Das Kerckhoffs'sche Prinzip	54
4.2	Mathematische Modellierung von Verschlüsselungsverfahren	55
4.3	Schlüsselaustausch	58
4.4	Angriffsszenarien	59
4.5	Brute-Force Angriffe	60
4.6	Wörterbuchangriffe (dictionary attacks)	62

4.1 Das Kerckhoffs'sche Prinzip

Ziel eines Verschlüsselungsverfahrens ist die Überführung einer im *Klartext* vorliegenden Nachricht in eine Nachricht in Form eines *Geheimtextes*, sodass aus einer Kenntnis des Geheimtextes keine (relevanten) Informationen über den Klartext gewonnen werden können (zumindest nicht innerhalb einer relevanten Zeitspanne und mit einem praktisch realisierbaren Aufwand).

Damit aus dem Geheimtext zu einem späteren Zeitpunkt der Klartext zurückgewonnen werden kann, muss zu dem Verschlüsselungsverfahren ein Entschlüsselungsverfahren korrespondieren, welches die Entschlüsselung eines Geheimtextes einer hierzu berechtigten Entität erlaubt.

Wie bereits in dem einführenden Kapitel 1 gesehen, ist es sicherlich keine gute Idee auf die Geheimhaltung von Ver- und Entschlüsselungsverfahren zu setzen. Zum einen können solche proprietäre Verfahren aufgrund der zu wahrenenden Geheimhaltung keinen breiten Begutachtungsprozess durch Experten durchlaufen, zum anderen sind bei der Nutzung solcher Verfahren diese so zu implementieren, dass diese nicht (z. B. durch ein Reverse-Engineering) offengelegt werden können.

Es gilt also heutzutage mehr denn je das bereits im Jahre 1883 von A. Kerckhoffs¹ formulierte Prinzip:

Kerckhoffs'sches Prinzip

Ein Verschlüsselungssystem darf nicht der Geheimhaltung bedürfen und soll ohne Schaden in Feindeshand fallen können.

Nicht von der Kenntnis des Verfahrens (dieses ist ja als bekannt vorauszusetzen), sondern von der Kenntnis eines Schlüssels muss die Möglichkeit zur Entschlüsselung einer Nachricht abhängen, d. h., es muss die folgende (sinngemäß auch bereits von Kerckhoffs formulierte) grundlegende Anforderung erfüllt sein:

Grundanforderung an Verschlüsselungsverfahren

Es soll nicht möglich sein, einen Geheimtext ohne Kenntnis des hierfür vorgesehenen Schlüssels effizient zu entschlüsseln.

Nach dieser sehr grundlegenden Anmerkung soll im folgenden Abschnitt eine mathematische Modellierung für Verschlüsselungsverfahren formuliert werden, mit der im Folgenden viele Aspekte von Verschlüsselungsverfahren präzise behandelt werden können.

Auf Grundlage dieser Modellierung kann beispielsweise die obige Grundanforderung an Verschlüsselungsverfahren in Abschnitt 4.5 in einer wesentlich konkreteren Form formuliert werden.

¹ Jean Guillaume Auguste Victor François Hubert Kerckhoffs von Nieuwenhof (*1835 Nuth, †1903 Paris) veröffentlichte unter Anderem das hier genannte Prinzip in seiner Arbeit *La cryptographie militaire*.

4.2 Mathematische Modellierung von Verschlüsselungsverfahren

Verschlüsselungsverfahren

Klartextnachrichten (plain text messages) und *Geheimtextnachrichten* (cipher text messages) sind Elemente einer Menge:

$$\mathcal{M}$$

Schlüssel sind Elemente einer als *Schlüsselraum* (key space) bezeichneten Menge:

$$\mathcal{K}$$

Ein *Verschlüsselungsverfahren* (auch *Chiffrierverfahren* (Cipher) genannt) besteht aus einem Paar von Funktionen (E, D) mit einer *Verschlüsselungsfunktion* E (E für encryption)

$$E : \mathcal{K} \times \mathcal{M} \rightarrow \mathcal{M} \quad (4.1)$$

und einer *Entschlüsselungsfunktion* D (D für decryption)

$$D : \mathcal{K} \times \mathcal{M} \rightarrow \mathcal{M} \quad (4.2)$$

wobei für jeden Schlüssel $k \in \mathcal{K}$ die durch

$$E_k : \mathcal{M} \rightarrow \mathcal{M}, \quad E_k(m) := E(k, m) \quad (4.3)$$

definierte Abbildung (Verschlüsselung mit dem Schlüssel k) bijektiv ist und die Umkehrfunktion $D_k := E_k^{-1}$ (Entschlüsselung mit dem Schlüssel k) durch

$$D_k : \mathcal{M} \rightarrow \mathcal{M}, \quad D_k(m) := D(k, m) \quad (4.4)$$

gegeben ist.

Kommentar Die Modellierung ist so allgemein gehalten, dass sie sowohl zur Beschreibung von *symmetrischen* als auch von *asymmetrischen* Verschlüsselungsverfahren geeignet ist.

Ein Verschlüsselungsverfahren heißt *asymmetrisch*, falls es zu jedem Schlüssel $k \in \mathcal{K}$ einen *öffentlichen Schlüssel* k_{pub} gibt, dessen Kenntnis zur Verschlüsselung aber nicht zur Entschlüsselung von Daten genügt. Die obigen Definitionen werden hierzu um die Menge der öffentlichen Schlüssel

$$\mathcal{K}_{pub}$$

und eine Funktion zur Ableitung der öffentlichen Schlüssel

$$\mathcal{K} \rightarrow \mathcal{K}_{pub}, \quad k \mapsto k_{pub}$$

sowie einer Verschlüsselungsfunktion

$$E_{pub} : \mathcal{K}_{pub} \times \mathcal{M} \rightarrow \mathcal{M}$$

ergänzt, wobei für alle $k \in \mathcal{K}, m \in \mathcal{M}$ gilt:

$$E_k(m) = E_{pub}(k_{pub}, m) =: E_{k_{pub}}(m)$$



Um die Anwendbarkeit der eingeführten Bezeichnungen zu illustrieren, betrachten wir den typischen Anwendungsfall, dass Alice eine vertrauenswürdige Nachricht m über einen unsicheren Kanal an Bob übertragen möchte. Hierzu haben sich Alice und Bob auf ein Verschlüsselungsverfahren (E, D) und einen Schlüssel k geeinigt. Diese Ausgangssituation ist in Abbildung 4.1 dargestellt.

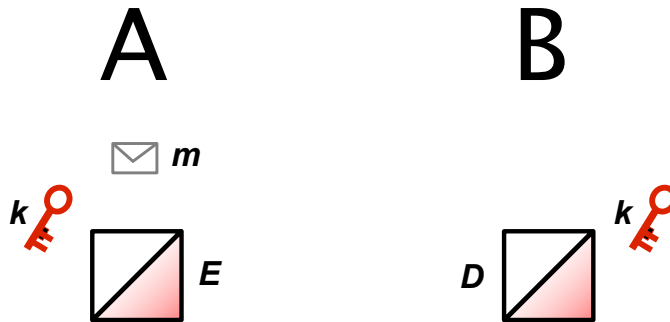


Abb. 4.1 Alice hat eine Nachricht $m \in \mathcal{M}$ für Bob. Alice und Bob haben sich auf ein Verschlüsselungsverfahren (E, D) und einen Schlüssel k geeinigt.

Unter Verwendung des Schlüssels k kann Alice die Klartextnachricht m mit der Funktion E_k verschlüsseln, d. h. die Geheimtextnachricht

$$c = E_k(m) = E(k, m)$$

berechnen. Diese kann dann über den unsicheren Kanal an Bob gesendet werden. Bob kann die Geheimtextnachricht c mit der Funktion D_k entschlüsseln und erhält so die Klartextnachricht m :

$$D(k, c) = D_k(c) = E_k^{-1}(c) = E_k^{-1}(E_k(m)) = m$$

Ver- und Entschlüsselung mit E_k bzw. D_k sind in Abbildung 4.2 dargestellt.

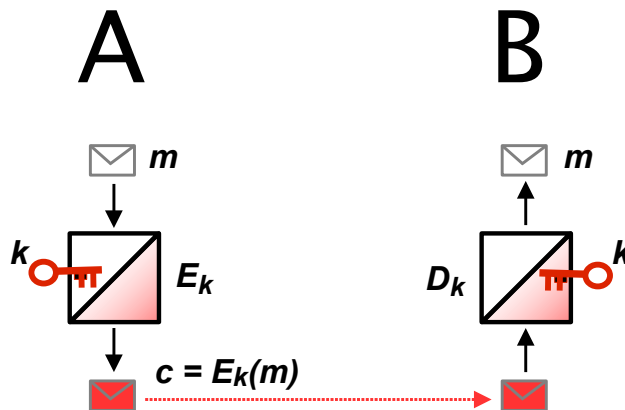


Abb. 4.2 Verschlüsselung der Klartextnachricht m mit E_k und Entschlüsselung der Geheimtextnachricht c mit D_k .

In der Praxis bestehen die Nachrichtenmengen $\mathcal{M}$ in der Regel aus endlichen Folgen (Tupeln) von Bit- oder Bytewerten, die entweder alle eine gleiche Länge besitzen, oder von beliebiger Länge sein können. D. h. mit $\mathcal{Z} = \mathbb{Z}_2$ oder $\mathcal{Z} = \mathbb{Z}_2^8$ gilt einer der folgenden Fälle:

- (i) $\mathcal{M} = \mathcal{Z}^n = \{(z_1, z_2, \dots, z_n) \mid z_i \in \mathcal{Z}\}$ für ein festes $n \in \mathbb{N}$
- (ii) $\mathcal{M} = \mathcal{Z}^{>0} := \bigcup_{n \in \mathbb{N}} \mathcal{Z}^n = \{(z_1, z_2, \dots, z_n) \mid z_i \in \mathcal{Z}, n \in \mathbb{N}\}$

Im ersten Fall wird von einem *Blockverschlüsselungsverfahren* (mit der *Blocklänge* n) gesprochen. Die Nachrichtenmengen aus (ii) treten dagegen in sogenannten *Stromverschlüsselungsverfahren* auf.

Bezeichnungen für Elemente aus $\mathbb{Z}_2^8$

Per Definition ist:

$$\mathbb{Z}_2^8 = \left\{ \begin{pmatrix} b_0 \\ b_1 \\ \vdots \\ b_7 \end{pmatrix} \mid b_i \in \mathbb{Z}_2 \right\}$$

Häufig werden Elemente aus $\mathbb{Z}_2^8$ als Bytewerte (Elemente aus $\mathbb{Z}_{256}$) notiert. Hierzu wird folgende bijektive Abbildung genutzt:

$$\begin{pmatrix} b_0 \\ b_1 \\ \vdots \\ b_7 \end{pmatrix} \mapsto \sum_{i=0}^7 b_i \cdot 2^i \quad (4.5)$$

Konkrete Bytewerte werden in der Regel als zweistellige Hexadezimalzahlen angegeben. (Auf ein vorgestelltes 0x wird im Folgenden meist verzichtet.)

Der Vektoraddition von Elementen aus $\mathbb{Z}_2^8$ entspricht die XOR-Operation von Bytewerten.

Frage 21

Geben Sie die Bytewerte an, die den Elementen

$$\begin{pmatrix} 1 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \end{pmatrix}, \begin{pmatrix} 0 \\ 1 \\ 1 \\ 1 \\ 0 \\ 1 \\ 0 \\ 1 \end{pmatrix} \in \mathbb{Z}_2^8$$

entsprechen.

4.3 Schlüsselaustausch

Möchte Alice eine verschlüsselte Nachricht über das Internet an Bob schicken, so müssen sich Alice und Bob zunächst *im Voraus* auf einen gemeinsamen Schlüssel verständigen (wie in den Abbildungen 4.1 und 4.2 dargestellt). Sicherlich wäre es keine gute Idee, wenn sie einen geheim zu haltenden Schlüssel über den gleichen Kanal im Klartext austauschten, der ja durch die Nutzung eines Verschlüsselungsverfahrens erst abgesichert werden soll!

Problem des Schlüsselaustausches

Bevor verschlüsselte Daten unter Verwendung eines symmetrischen Verschlüsselungsverfahrens ausgetauscht werden können, müssen die Kommunikationspartner auf vertrauenswürdige Weise in den Besitz eines gemeinsamen Schlüssels kommen.

Wie könnten Schlüssel im Voraus über einen (hoffentlich) sicheren Kanal ausgetauscht werden? Einige Beispiele (deren Praktikabilität Sie für sich selbst bewerten sollten) seien genannt:

- Der Schlüssel wird per Brief von Bob an Alice geschickt.
- Alice und Bob einigen sich per Telefonat auf einen Schlüssel.
- Bob übergibt Alice eine Kopie des Schlüssels auf einem elektronischen Datenträger (CD, Memorystick, Chipkarte).

Auch wenn die oben genannten Offline-Lösungen in speziellen Anwendungen sinnvoll einsetzbar sein mögen, schließen sie eine Nutzung in Anwendungen aus, die ohne Aufwand eine sichere Verbindung zwischen Kommunikationspartnern nutzen wollen, die zuvor noch keine Schlüssel ausgetauscht haben. Als ein äußerst populäres Beispiel seien verschlüsselte HTTPS-Verbindungen zwischen Webbrowsern und Webservern genannt. Offensichtlich wäre eine Speicherung und das Management von Secret Keys für alle möglichen Browser-Server-Verbindungen eine kaum zu bewältigende Aufgabe!

Beispiel 4.1

Ein auf symmetrischen Verschlüsselungsverfahren basierender Online-Lösungsansatz besteht in der Etablierung einer vertrauenswürdigen Instanz, nennen wir sie Trudy, zur Weiterleitung oder Verteilung von Schlüsseln. Unter der Voraussetzung, dass jeder potentielle Kommunikationspartner X mit Trudy (T) einen spezifischen Schlüssel k_{TX} teilt, kann Bob (B) einen Schlüssel k_{AB} via Trudy an Alice senden: Bob schickt $c_{TB} = E_{k_{TB}}(k_{AB})$ an Trudy. Trudy entschlüsselt c_{TB} mit k_{TB} und erhält so den Schlüssel k_{AB} , den sie dann mit k_{TA} verschlüsselt an Alice schickt. Alice erhält also $c_{TA} = E_{k_{TA}}(k_{AB})$ von Trudy und nach Entschlüsselung mit k_{TA} den Schlüssel k_{AB} .

Die Vertrauenswürdigkeit von Trudy ist bei dem auf symmetrischen Verfahren basierenden Schlüsselaustauschverfahren natürlich fundamental, da Trudy bei der Übermittlung auch Kenntnis vom Schlüssel k_{AB} erhält. Zusätzlich muss Trudy für jeden Kommunikationspartner einen Schlüssel vorhalten und sicher speichern. Schließlich ist die Verfügbarkeit des von Trudy angebotenen Dienstes von essenzieller Bedeutung für alle weiteren Kommunikationspartner. ◀

Glücklicherweise gibt es auch Online-Verfahren, bei denen der auszutauschende Schlüssel k_{AB} nicht durch die Hände Dritter gehen muss. Diese basieren auf asymmetrischen kryptographischen Verfahren und werden im Studienheft 03 ausführlich behandelt.

4.4 Angriffsszenarien

In der in Abschnitt 4.1 formulierten Grundanforderung an Verschlüsselungsverfahren, dass eine effiziente Entschlüsselung eines Geheimtextes ohne Kenntnis des hierfür vorgesehenen Schlüssels nicht möglich sein soll, werden keine weitere Einschränkungen gemacht, unter welchen Voraussetzungen diese Anforderung erfüllt sein soll.

Die verschiedenen Voraussetzungen, die einer Angreiferin möglicherweise zur Verfügung stehen, um aus einer gegebenen Geheimtextnachricht $c = E_k(m)$ ohne Kenntnis von k die Klartextnachricht m zu bestimmen, sollen daher im Folgenden konkretisiert werden.

Beachten Sie zunächst, dass ein erfolgreicher Angriff auf eine Geheimtextnachricht nicht so stark ist, wie ein erfolgreicher Angriff auf den bei der Verschlüsselung benutzten Schlüssel k : Ist k erst einmal bekannt, so kann ja jede Geheimtextnachricht sofort entschlüsselt werden. (Tatsächlich zielen die meisten Angriffe auf die Bestimmung des eingesetzten Schlüssels ab.)

Ciphertext-only Angriffe (known-ciphertext Angriffe)

Voraussetzung: Der Geheimtext $c = E_k(m)$ und möglicherweise weitere Geheimtexte $c_i = E_k(m_i)$ sind verfügbar.

Angriffsziel: Bestimmung von m oder (besser noch) von k .

Bessere Voraussetzungen als bei einem ciphertext-only Angriff hat ein Angreifer bei einem known-plaintext Angriff, da hier nicht nur Geheimtextnachrichten sondern auch zugehörige Klartextnachrichten zur Verfügung stehen:

Known-plaintext Angriffe

Voraussetzung: Der Geheimtext $c = E_k(m)$ und eine Reihe von Paaren $(m_i, E_k(m_i))$ sind verfügbar.

Angriffsziel: Bestimmung von m oder (besser noch) von k .

Noch bessere Voraussetzungen als bei einem known-plaintext Angriff hat ein Angreifer bei einem chosen-plaintext Angriff, da er hier zu Klartextnachrichten seiner Wahl die zugehörigen Geheimtextnachrichten erhalten kann:

Chosen-plaintext Angriffe

Voraussetzung: Der Geheimtext $c = E_k(m)$ ist gegeben und für eine Reihe beliebig vorgegebener Klartextnachrichten m_i können die zugehörigen Geheimtextnachrichten $c_i = E_k(m_i)$ erhalten werden.

Angriffsziel: Bestimmung von m oder (besser noch) von k .

Auf Grundlage der Klassifikation über die in Betracht zu ziehenden Möglichkeiten, die einem potentiellen Angreifer zur Verfügung stehen könnten, kann die in Abschnitt 4.1 formulierte Grundanforderung etwas präziser gefasst werden:

Grundanforderung an Verschlüsselungsverfahren (Version 2)

Die Durchführung eines chosen-plaintext Angriffs soll nicht effizient möglich sein.

Die bisher beschriebenen Angriffstypen betreffen die theoretischen (insbesondere algorithmischen) Aspekte der jeweiligen Verschlüsselungsverfahren. Grundlage solcher Angriffe bilden daher vertiefte Analysen der zum Einsatz kommenden Ver- und Entschlüsselungsfunktionen.

Von ganz anderer Art sind Angriffe, die nicht auf mögliche Schwächen der Verschlüsselungsverfahren abzielen, sondern Schwachstellen oder Fehler ihrer Implementierungen ausnutzen. In diesem Zusammenhang sind auch sogenannte *ciphertext-only* Angriffe von Bedeutung, bei denen die Reaktionen (Fehlermeldungen, Zeitverhalten) realer Implementierungen von Verschlüsselungsverfahren auf ausgewählte Geheimtextnachrichten analysiert werden.

4.5 Brute-Force Angriffe

Die im letzten Abschnitt 4.4 zugeschärfte Formulierung der Grundanforderung aus Abschnitt 4.1 an Verschlüsselungsverfahren soll in diesem Abschnitt nochmals weiter präzisiert werden. Möglicherweise hat es Sie bereits gestört, dass dem Wörtchen *effizient* in beiden Formulierungen eine gewichtige, aber nicht weiter erklärte Bedeutung zukommt. Es soll daher im Folgenden geklärt werden, was *Effizienz* in diesem Zusammenhang genau bedeutet. Hierzu machen wir uns zunächst klar, dass es im Prinzip immer möglich ist, einen Angriff auf den in einem Verschlüsselungsverfahren benutzten Schlüssel k durchzuführen:

Brute-Force Angriffe auf k

Liegen die Voraussetzungen eines known-plaintext Angriffs vor, so kann k gefunden werden, indem alle Schlüssel des Schlüsselraums durchlaufen und „ausprobiert“ werden:

Für die nach Voraussetzung vorliegenden Paare (m_i, c_i) mit $c_i = E_k(m_i)$ bezeichne $\mathcal{K}_i$ die Menge der jeweils „passenden“ Schlüssel:

$$\mathcal{K}_i := \{\tilde{k} \in \mathcal{K} \mid E_{\tilde{k}}(m_i) = c_i\}$$

Indem alle Schlüssel des Schlüsselraums einmal durchlaufen werden, ergibt sich für eine (in praktischen Anwendungen kleine und sich dynamisch beim Durchlauf ergebende) Anzahl N von Paaren $(m_i, E_k(m_i))$:

$$\bigcap_{i=1}^N \mathcal{K}_i = \{k\}$$

D. h. k ist durch die Eigenschaft bestimmt, dass k der einzige Schlüssel ist, der zu den N Klartext-Geheimtext-Paaren $(m_i, E_k(m_i))$ für $i = 1, \dots, N$ passt.

Werden die Schlüssel des Schlüsselraums in einem Brute-Force Angriff durchlaufen, so wird der richtige Schlüssel spätestens nach $|\mathcal{K}|$ und durchschnittlich nach $\frac{|\mathcal{K}|}{2}$ Schritten (Erwartungswert) gefunden, was durch einen Test mit wenigen weiteren Klartext-Geheimtext-Paaren mit an Sicherheit grenzender Wahrscheinlichkeit bestätigt werden kann, ohne die restlichen Schlüssel testen zu müssen. Mit Bezug auf einen Brute-Force Angriff auf k , kann die Grundanforderung an Verschlüsselungsverfahren also dahingehend präzisiert werden, dass ein Angriff zur Bestimmung von k durchschnittlich mindestens $\frac{|\mathcal{K}|}{2}$ Ver- oder Entschlüsselungsoperationen benötigt.

Frage 22

Sei

$$E : \mathcal{K} \times \mathcal{M} \rightarrow \mathcal{M}$$

ein Blockverschlüsselungsverfahren mit:

$$K := |\mathcal{K}| \quad \text{und} \quad M := |\mathcal{M}|$$

Betrachten Sie die Menge

$$\mathcal{E} := \{E_k \mid k \in \mathcal{K}\}$$

wie eine Menge von K zufällig gewählten Permutationen auf $\mathcal{M}$ und zeigen Sie:

- (i) Zu $m, c \in \mathcal{M}$ gibt es durchschnittlich $\frac{K}{M}$ viele Schlüssel $k \in \mathcal{K}$ mit $E_k(m) = c$.
- (ii) Sind $m, c \in \mathcal{M}$ zufällig gewählt, so gibt es mit der Wahrscheinlichkeit

$$p = 1 - \left(1 - \frac{1}{M}\right)^K$$

mindestens einen Schlüssel $k \in \mathcal{K}$ mit $E_k(m) = c$.

- (iii) Gilt $K = M$, so gibt es zu zufällig gewählten $m, c \in \mathcal{M}$ mit der Wahrscheinlichkeit

$$p = 1 - \left(1 - \frac{1}{M}\right)^M \approx 1 - e^{-1} \approx 0,632$$

mindestens einen Schlüssel $k \in \mathcal{K}$ mit $E_k(m) = c$.

- (iv) Gilt $K \ll M$, so gibt es zu zufällig gewählten $m, c \in \mathcal{M}$ mit einer Wahrscheinlichkeit p mindestens einen Schlüssel $k \in \mathcal{K}$ mit $E_k(m) = c$. Hierbei gilt:

$$p \approx 1 - e^{-\frac{K}{M}} \approx \frac{K}{M}$$

Kommentar Für den häufig vorkommenden Fall, dass alle Klartextnachrichten, die im Rahmen einer Anwendung auftreten können, Elemente einer relativ kleinen Teilmenge $\mathcal{M}_0$ der Nachrichtenmenge $\mathcal{M}$ sind, kann der Brute-Force Angriff zu einem ciphertext-only Angriff abgeändert werden, indem für die Reihe der vorliegenden Geheimtextnachrichten c_i die Mengen $\mathcal{K}_i$ nun folgendermaßen definiert werden:

$$\mathcal{K}_i := \{\tilde{k} \in \mathcal{K} \mid D_{\tilde{k}}(c_i) \in \mathcal{M}_0\}$$

Gelingt ein Angriff auf den Schlüssel k , so kann auch zu jeder Geheimtextnachricht c die zugehörige Klartextnachricht m ohne weiteren Aufwand durch Entschlüsselung bestimmt werden. Für den Fall, dass die Anzahl möglicher Klartextnachrichten $|\mathcal{M}|$ kleiner ist als die Anzahl der möglichen Schlüssel $|\mathcal{K}|$, werden auch direkte Angriffe auf m relevant, da diese dann effizienter sind als Angriffe auf k .

Brute-Force Angriffe auf m

Liegen die Voraussetzungen eines chosen-plaintext Angriffs vor, so kann zu der Geheimtextnachricht $c = E_k(m)$ die zugehörige Klartextnachricht m gefunden werden, indem die möglichen Klartextnachrichten $\tilde{m} \in \mathcal{M}$ nacheinander durchlaufen und „ausprobiert“ werden, bis m gefunden ist:

$$E_k(\tilde{m}) = c = E_k(m) \implies \tilde{m} = m$$

D. h. m wird durch die Eigenschaft bestimmt, dass m die einzige Klartextnachricht mit $E_k(m) = c$ ist.

Werden die Nachrichten aus $\mathcal{M}$ in einem Brute-Force Angriff durchlaufen, so wird die richtige Klartextnachricht spätestens nach $|\mathcal{M}|$ und durchschnittlich nach $\frac{|\mathcal{M}|}{2}$ Schritten (Erwartungswert) gefunden. Mit Bezug auf einen Brute-Force Angriff auf m , kann die Grundanforderung an Verschlüsselungsverfahren also dahingehend zusätzlich präzisiert werden, dass ein Angriff zur Bestimmung von m durchschnittlich mindestens $\frac{|\mathcal{M}|}{2}$ Ver- oder Entschlüsselungsoperationen benötigt.

Grundanforderung an Verschlüsselungsverfahren (Version 3)

Die Durchführung eines chosen-plaintext Angriffs soll nicht effizient möglich sein, d. h.:

- (i) Es soll keinen Angriff auf den Schlüssel k geben, der durchschnittlich weniger als $\frac{|\mathcal{K}|}{2}$ Ver- oder Entschlüsselungsoperationen benötigt.
- (ii) Es soll keinen Angriff auf eine Klartextnachricht m geben, der durchschnittlich weniger als $\min\left\{\frac{|\mathcal{K}|}{2}, \frac{|\mathcal{M}|}{2}\right\}$ Ver- oder Entschlüsselungsoperationen benötigt.

4.6 Wörterbuchangriffe (dictionary attacks)

Werden unter Verwendung eines Schlüssels k eine Reihe von Nachrichten $m_1, m_2, m_3, \dots$ verschlüsselt, so lässt sich an der Folge der entsprechenden Geheimtextnachrichten $c_1, c_2, c_3, \dots$ natürlich ablesen, ob Klartextnachrichten wiederholt auftreten, denn es gilt ja:

$$c_i = c_j \quad \Longleftrightarrow \quad m_i = m_j$$

Möglicherweise gelingt es einer Angreiferin aber sogar für eine Reihe von Geheimtextnachrichten Kenntnis über die zugehörigen Klartextnachrichten zu erlangen (wie in den Voraussetzungen eines known-plaintext Angriffs). Diese Klartext-Geheimtext-Nachrichtenpaare können sortiert nach den Geheimtexten im Sinne eines Wörterbuchs vorgehalten werden, und für den Fall, dass eine der in dem Wörterbuch vorkommenden Geheimtextnachrichten erneut beobachtet wird, kann die entsprechende Klartextnachricht dem Wörterbuch entnommen werden.

Liegen die Voraussetzungen des chosen-plaintext Angriffs vor, so kann eine Angreiferin für eine Teilmenge $\mathcal{M}_0$ von Klartextnachrichten, deren Auftreten wahrscheinlich oder von besonderem Interesse ist, ein Wörterbuch mit den Einträgen

$$(E_k(m), m) \quad \text{für } m \in \mathcal{M}_0$$

anlegen.

Ein wirksames Mittel zum Schutz vor Wörterbuchangriffen besteht darin, jede Nachricht vor einer Verschlüsselung auf eine spezifische Weise reversibel abzuändern. Bei einer wiederholten Verschlüsselung der gleichen Klartextnachricht wird diese dann jeweils unterschiedlich abgeändert und in der Regel zu unterschiedlichen Geheimtextnachrichten verschlüsselt. Um nach der Entschlüsselung einer Geheimtextnachricht die Abänderung der Klartextnachricht wieder rückgängig machen zu können, wird gegebenenfalls eine entsprechende Information benötigt, die zusammen mit dem Geheimtext zu übertragen ist.

Die oben beschriebene Abänderung einer Klartextnachricht wird üblicherweise durch einen (Zahlen-)Wert gesteuert. Je zwei Werte, die zur Abänderung von Klartextnachrichten genutzt werden, müssen dann also auch verschieden sein. Die Werte sind daher sogenannte Nonces:

Nonce (Number used Once)

Parameter, die bei einer wiederholten Anwendung im Rahmen eines kryptographischen Verfahrens, stets unterschiedliche Werte besitzen sollen, werden *Nonce (Number used Once)* genannt.

Typische Beispiele zur Generierung von Nonce-Werten:

- *Zähler (Counter)*, also eine ganzzahlige Variablen, die nach jeder Nutzung als Nonce-Wert inkrementiert wird.
- *Zeitstempel (Time Stamp)* mit einer hinreichend feinen Zeitauflösung.
- Zufallszahlen aus einem hinreichend großen Zahlbereich, so dass das wiederholte Auftreten einer Zahl im Rahmen der jeweiligen Anwendung mit an Sicherheit grenzender Wahrscheinlichkeit ausgeschlossen werden kann.

Beispiel 4.2 (Schutz vor Wörterbuchangriffen für $\mathcal{M} = \mathcal{Z}^n$)

Für $\mathcal{Z} = \mathbb{Z}_2$ oder $\mathcal{Z} = \mathbb{Z}_2^8$ bildet $\mathcal{M} = \mathcal{Z}^n$ einen Vektorraum mit der Möglichkeit zwei Elemente aus $\mathcal{M}$ zu addieren (komponentenweise XOR-Operation).

Zum Schutz vor Wörterbuchangriffen können Nonce-Werte ebenfalls aus $\mathcal{M}$ gewählt werden. Eine Klartextnachricht $m \in \mathcal{M}$ wird von Alice mit einem Nonce-Wert $v \in \mathcal{M}$ abgeändert zu

$$\tilde{m} = m + v$$

und verschlüsselt zu:

$$c = E_k(\tilde{m})$$

Um die zu c gehörige Klartextnachricht m bestimmen zu können, benötigt Bob nicht nur den Schlüssel k sondern auch den Nonce-Wert v :

$$m = D_k(c) + v$$

In vielen Anwendungen wird v zusammen mit c an Bob übertragen. (Möglich ist aber beispielsweise auch, dass v den aktuellen Wert von Zählervariablen repräsentiert, die von Alice und Bob synchron gehalten werden.) ◀

